

SORTING — COMPLETE GUIDE

A comprehensive placement + CP focused guide to every sorting algorithm and concept you need.

TABLE OF CONTENTS

1. [Selection Sort](#)
 2. [Bubble Sort](#)
 3. [Insertion Sort](#)
 4. [Merge Sort](#)
 5. [Quick Sort](#)
 6. [Counting Sort](#)
 7. [Custom Comparator](#)
 8. [Sorting Intervals](#)
 9. [Stable vs Unstable Sort](#)
 10. [Radix Sort](#)
 11. [Bucket Sort](#)
 12. [Topological Sorting](#)
 13. [Sort + Greedy](#)
 14. [External Sorting](#)
 15. [TimSort Idea](#)
-

1. SELECTION SORT

1. Overview

Selection Sort is a simple comparison-based sorting algorithm. It repeatedly selects the smallest (or largest) element from the unsorted portion of the array and swaps it into its correct position at the beginning of the sorted portion.

2. Intuition

Simple explanation: Imagine you have a deck of face-down cards. You look through all of them, pick the smallest one, and place it first. Then you look through the remaining cards, pick the next smallest, and place it second. Repeat until all cards are sorted.

Analogy: Like a talent show judge picking the best contestant first, then the second-best from the remaining, and so on.

Step-by-step reasoning:

1. Find the minimum element in the entire array.
2. Swap it with the element at index 0.

3. Find the minimum in the remaining array (indices 1 to $n-1$).
4. Swap it with the element at index 1.
5. Continue until the entire array is sorted.

Why it works: Each pass places one element in its final sorted position. After k passes, the first k elements are in their correct positions. After $n-1$ passes, the array is fully sorted.

3. When to Use It

- When the array is very small ($n \leq 20-30$).
- When minimizing the number of swaps is important (it makes at most $n-1$ swaps).
- When memory is extremely limited and you need an in-place sort.
- When simplicity is valued over performance.
- When you need a stable sort is **not** required.

Trigger phrases:

- "Sort with minimum swaps"
- "In-place sort"
- "Simple sorting"

4. When Not to Use It

- **Never in production** — $O(n^2)$ time makes it unusable for large inputs.
- When the array is large ($n > 1000$).
- When the input is nearly sorted — it still takes $O(n^2)$ time.
- When stability is required (Selection Sort is not stable by default).
- When faster algorithms like Merge Sort, Quick Sort, or built-in sort are available.

Simpler alternatives: Insertion Sort (better for nearly sorted data), built-in `sort()`.

5. Core Concepts

Minimum Selection

Each pass scans the unsorted portion to find the minimum element. This is the core operation.

Swapping

After finding the minimum, it is swapped with the first element of the unsorted portion. This is why the number of swaps is at most $n-1$.

Sorted/Unsorted Boundary

The array is conceptually divided into a sorted prefix (left side) and an unsorted suffix (right side). The boundary moves right by one each pass.

In-Place

Selection Sort sorts the array without requiring extra memory — $O(1)$ auxiliary space.

6. Step-by-Step Algorithm

```
Input:  arr[0..n-1]

1. For i = 0 to n-2:
  a. Set minIdx = i
  b. For j = i+1 to n-1:
    If arr[j] < arr[minIdx], set minIdx = j
  c. If minIdx != i, swap arr[i] and arr[minIdx]
```

7. Dry Run

Input: [64, 25, 12, 22, 11]

Pass	i	Array State (before swap)	minIdx	Swapped With	Array State (after swap)
1	0	[64, 25, 12, 22, 11]	4	arr[0]	[11, 25, 12, 22, 64]
2	1	[11, 25, 12, 22, 64]	2	arr[1]	[11, 12, 25, 22, 64]
3	2	[11, 12, 25, 22, 64]	3	arr[2]	[11, 12, 22, 25, 64]
4	3	[11, 12, 22, 25, 64]	3	no swap	[11, 12, 22, 25, 64]

Final Sorted Array: [11, 12, 22, 25, 64]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        // Find the index of the minimum element in unsorted portion
        int minIdx = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        // Swap if needed
        if (minIdx != i) {
            swap(arr[i], arr[minIdx]);
        }
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
```

```
selectionSort(arr);
for (int x : arr) cout << x << " ";
// Output: 11 12 22 25 64
return 0;
}
```

9. Python Implementation

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        # Find the index of the minimum element in unsorted portion
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        # Swap if needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

arr = [64, 25, 12, 22, 11]
selection_sort(arr)
print(arr) # [11, 12, 22, 25, 64]
```

10. Code Explanation

- **Outer loop** ($i = 0$ to $n-2$): Tracks the boundary between sorted and unsorted portions. Each iteration places one element in its correct position.
- **Inner loop** ($j = i+1$ to $n-1$): Scans the unsorted portion to find the index of the minimum element.
- **minIdx**: Tracks the index of the current minimum found during the scan.
- **swap(arr[i], arr[minIdx])**: Places the minimum element at the boundary position.
- **if (minIdx != i)**: Avoids an unnecessary swap if the element is already in the correct position.

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n^2)$
Time (Worst)	$O(n^2)$
Time (Average)	$O(n^2)$
Space	$O(1)$
Swaps	$O(n)$
Comparisons	$O(n^2)$

Operation	Complexity
Stable	No

12. Common Patterns

Minimum Swaps to Sort

- **How to identify:** Problem asks for minimum number of swaps to sort an array.
- **Approach:** Use cycle detection on the permutation graph.
- **Example:** GFG "Minimum Swaps to Sort"

Partially Sorting First K Elements

- **How to identify:** Need only the first k smallest elements.
- **Approach:** Run Selection Sort for k passes.
- **Example:** "Find K Smallest Elements" (partial selection sort)

13. Common Mistakes

- **Off-by-one in outer loop:** Running $i < n$ instead of $i < n-1$ — the last element is already sorted when $n-1$ elements are placed.
- **Resetting `minIdx` incorrectly:** Always set `minIdx = i` at the start of each pass.
- **Swapping even when `minIdx == i`:** Unnecessary swap wastes time.
- **Assuming stability:** Selection Sort is not stable; swapping can change relative order of equal elements.

14. Edge Cases

- **Empty array:** No swaps needed, outer loop doesn't execute.
- **Single element:** No swaps needed, outer loop doesn't execute.
- **Already sorted:** Still $O(n^2)$ comparisons, but no swaps.
- **Reverse sorted:** Maximum swaps $(n-1)$, still $O(n^2)$.
- **All equal elements:** No swaps if we check `minIdx != i`.

15. Variations

Bidirectional Selection Sort (Cocktail Sort)

- Selects both min and max in each pass, reducing passes by $\sim$ half.
- Same complexity, slightly better constants.
- Not important for placements.

Heap Sort Variant

- Replaces the linear scan with a heap for $O(n \log n)$ time.
- This is essentially Heap Sort, not Selection Sort anymore.

16. Related Algorithms/Data Structures

- **Insertion Sort:** Also $O(n^2)$ but adaptive (better for nearly sorted data). Selection Sort is not adaptive.

- **Bubble Sort:** Also $O(n^2)$ but Bubble Sort is stable and can be optimized to stop early.
- **Heap Sort:** Uses the "selection" idea but with a heap for $O(n \log n)$. This is the optimized version of Selection Sort.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (use Selection Sort for practice)
- **Selection Sort** — GFG (implement and test)

Medium

- **Minimum Swaps to Sort** — GFG (cycle detection + selection idea)
- **Kth Smallest Element** — LeetCode 215 (QuickSelect preferred, but partial Selection Sort works)

Hard

- **Sorting with Swaps (Restore the Array)** — Codeforces (permutation cycles)

18. Interview Explanation

"Selection Sort repeatedly finds the minimum element from the unsorted part and moves it to the beginning. It maintains two subarrays: sorted and unsorted. In each pass, it scans the unsorted portion to find the minimum, then swaps it into place. It runs in $O(n^2)$ time for all cases, uses $O(1)$ space, and makes at most $n-1$ swaps. It's not stable. It's mainly useful for small arrays or when swap cost is high."

19. Revision Notes

- Each pass places one element at its correct position.
- Always $O(n^2)$ comparisons — no best-case improvement.
- $O(n)$ swaps — minimum among $O(n^2)$ sorts.
- Not stable.
- Never use in production.

20. Final Cheat Sheet

Property	Value
When to use	Tiny arrays, min swaps
Type	Comparison, in-place
Time	$O(n^2)$
Space	$O(1)$
Stable	No
Key code idea	<code>minIdx = j; swap(arr[i], arr[minIdx])</code>
Edge cases	Empty, single, duplicate, sorted

2. BUBBLE SORT

1. Overview

Bubble Sort repeatedly steps through the array, compares adjacent elements, and swaps them if they are in the wrong order. Larger elements "bubble up" to the end of the array with each pass.

2. Intuition

Simple explanation: Imagine bubbles in water. Larger bubbles rise to the top faster. Similarly, larger elements in the array "bubble up" to their correct position at the end.

Analogy: Like standing in a line and repeatedly asking the person next to you to swap if you're taller. After enough passes, the tallest person ends up at the end.

Step-by-step reasoning:

1. Compare each adjacent pair from left to right.
2. If the left element is greater than the right, swap them.
3. After one full pass, the largest element is at the end.
4. Repeat for the remaining $n-1$ elements, ignoring the last sorted position.
5. If no swaps occur in a pass, the array is already sorted — we can stop early.

Why it works: Each pass pushes the largest unsorted element to its correct position at the end. After k passes, the last k elements are sorted. After at most $n-1$ passes, the array is sorted.

3. When to Use It

- For educational purposes (learning sorting basics).
- When the array is very small.
- When the array is nearly sorted (with the optimized early-exit version).
- When you need a simple stable sort.
- When you want to detect if an array is already sorted (single pass, no swaps).

Trigger phrases:

- "Stable simple sort"
- "Check if array is sorted"
- "Nearly sorted array"

4. When Not to Use It

- **Never in production** — $O(n^2)$ is too slow for any real-world use.
- When the array is large.
- When performance matters.
- When you need $O(n \log n)$ or better.
- When the array is reverse sorted (worst case).

Simpler alternatives: Insertion Sort (better constant factors), built-in `sort()`.

5. Core Concepts

Adjacent Comparison and Swap

The fundamental operation: compare two neighboring elements and swap if out of order.

Passes

Each pass scans from left to right. After each pass, one more element is in its final position at the end.

Optimization: Early Exit

If a pass completes with no swaps, the array is sorted. This makes Bubble Sort adaptive — $O(n)$ for already sorted arrays.

Stable

Equal elements are not swapped across each other, so Bubble Sort is naturally stable.

6. Step-by-Step Algorithm

```
Input: arr[0..n-1]

1. For i = 0 to n-2:
  a. Set swapped = false
  b. For j = 0 to n-2-i:
    If arr[j] > arr[j+1]:
      Swap arr[j] and arr[j+1]
      Set swapped = true
  c. If swapped == false, break
```

7. Dry Run

Input: [5, 1, 4, 2, 8]

Pass 1:

j	Comparison	Swap?	Array
0	5 > 1	Yes	[1, 5, 4, 2, 8]
1	5 > 4	Yes	[1, 4, 5, 2, 8]
2	5 > 2	Yes	[1, 4, 2, 5, 8]
3	5 > 8	No	[1, 4, 2, 5, 8]

Pass 2:

j	Comparison	Swap?	Array
0	1 > 4	No	[1, 4, 2, 5, 8]
1	4 > 2	Yes	[1, 2, 4, 5, 8]
2	4 > 5	No	[1, 2, 4, 5, 8]

Pass 3:

j	Comparison	Swap?	Array
0	1 > 2	No	[1, 2, 4, 5, 8]
1	2 > 4	No	[1, 2, 4, 5, 8]

No swaps → break.

Final: [1, 2, 4, 5, 8]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        bool swapped = false;
        // Last i elements are already in place
        for (int j = 0; j < n - 1 - i; ++j) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        // If no swaps, array is sorted
        if (!swapped) break;
    }
}

int main() {
    vector<int> arr = {5, 1, 4, 2, 8};
    bubbleSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 1 2 4 5 8
    return 0;
}
```

9. Python Implementation

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

arr = [5, 1, 4, 2, 8]
bubble_sort(arr)
print(arr)  # [1, 2, 4, 5, 8]
```

10. Code Explanation

- **Outer loop** ($i = 0$ to $n-2$): Controls the number of passes. At most $n-1$ passes needed.
- **swapped flag**: If set to `false` at start of pass and never set to `true`, the array is sorted and we break.
- **Inner loop** ($j = 0$ to $n-2-i$): The $-i$ is important — after i passes, the last i elements are already sorted, so we don't need to check them.
- **swap(arr[j], arr[j+1])**: Swaps adjacent elements if they are out of order.

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n)$
Time (Worst)	$O(n^2)$
Time (Average)	$O(n^2)$
Space	$O(1)$
Swaps	$O(n^2)$
Stable	Yes

12. Common Patterns

Detect if Array is Sorted

- **Approach**: One pass of Bubble Sort without swaps → if no swaps, array is sorted.
- **Complexity**: $O(n)$.

Sort Nearly Sorted Array

- **Approach**: If each element is at most k positions away from its correct position, Bubble Sort can sort in $O(n \cdot k)$.

- **Note:** Insertion Sort is better for this pattern.

13. Common Mistakes

- **Forgetting `-i` in inner loop:** Leads to unnecessary comparisons.
- **Not using early exit:** Without the `swapped` flag, best case is $O(n^2)$ instead of $O(n)$.
- **Wrong direction:** Comparing `arr[j] > arr[j+1]` for ascending; using `<` gives descending.
- **Off-by-one:** Inner loop `j < n-1-i` (not `j < n-i`).

14. Edge Cases

- **Empty array:** No passes, no swaps.
- **Single element:** No passes, no swaps.
- **Already sorted:** One pass, no swaps $\rightarrow O(n)$.
- **Reverse sorted:** Maximum passes and swaps $\rightarrow O(n^2)$.
- **All equal:** No swaps $\rightarrow O(n)$ with early exit.
- **Large values:** No overflow risk (only comparisons, no arithmetic).

15. Variations

Cocktail Shaker Sort (Bidirectional Bubble Sort)

- Alternates between left-to-right and right-to-left passes.
- Slightly better for arrays where small elements are at the end.
- Same $O(n^2)$ complexity.

Comb Sort

- Uses a gap larger than 1, shrinking over time.
- Improves Bubble Sort to $O(n \log n)$ average.
- Rarely used in practice.

16. Related Algorithms/Data Structures

- **Insertion Sort:** Also $O(n^2)$ and stable. Insertion Sort is generally better (fewer comparisons on average).
- **Selection Sort:** $O(n^2)$ but not stable. Selection Sort makes fewer swaps ($O(n)$ vs $O(n^2)$).
- **Quick Sort:** Uses partitioning which is related to "bubbling" but much more efficient.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Bubble Sort for practice)
- **Bubble Sort** — GFG (implement and test)

Medium

- **Minimum Number of Swaps to Sort** — no direct relation, but Bubble Sort swap count = inversion count for adjacent swaps.

- **Count Inversions in Array** — GFG (Bubble Sort swap count = inversion count, but use Merge Sort for efficiency)

Hard

- **Sorting the Sentence** — LeetCode 1859 (trivial, but helps understand stable sorting)

18. Interview Explanation

"Bubble Sort repeatedly steps through the array, comparing adjacent elements and swapping them if they're in the wrong order. Each pass pushes the largest element to the end. With an optimization that stops early if no swaps occur, it runs in $O(n)$ for an already sorted array and $O(n^2)$ in the worst case. It's stable, in-place, and uses $O(1)$ space. It's mainly used for educational purposes."

19. Revision Notes

- Compare adjacent, swap if out of order.
- Largest element "bubbles" to end each pass.
- Early exit: if no swaps in a pass → sorted.
- $O(n)$ best, $O(n^2)$ worst, $O(1)$ space.
- Stable.

20. Final Cheat Sheet

Property	Value
When to use	Small/nearly sorted arrays, education
Type	Comparison, in-place, stable
Time	$O(n)$ best, $O(n^2)$ worst
Space	$O(1)$
Stable	Yes
Key code idea	<code>if(arr[j] > arr[j+1]) swap();</code>
Edge cases	Empty, single, sorted, reverse, duplicates

3. INSERTION SORT

1. Overview

Insertion Sort builds the sorted array one element at a time by repeatedly picking the next element and inserting it into its correct position among the already sorted elements.

2. Intuition

Simple explanation: Like sorting a hand of playing cards. You pick up one card at a time and insert it into its correct position among the cards you're already holding, shifting the other cards to make room.

Analogy: Imagine arranging books on a shelf by height. You take one book at a time and slide it into the correct position, pushing taller books to the right.

Step-by-step reasoning:

1. Start with the first element — it's already "sorted" by itself.
2. Take the next element (the "key").
3. Compare it with elements in the sorted portion (moving right to left).
4. Shift elements greater than the key one position to the right.
5. Insert the key into the empty slot.
6. Repeat for all remaining elements.

Why it works: After processing k elements, the first k elements are sorted. Each new element is inserted into its correct position, maintaining the sorted invariant.

3. When to Use It

- When the array is small ($n \leq 50$).
- When the array is **nearly sorted** — Insertion Sort is adaptive and runs in $O(n)$ for nearly sorted data.
- When elements are added incrementally and you need to maintain a sorted list online.
- When you need a stable, in-place sort.
- When you want a simple sort that works well in practice for small n .

Trigger phrases:

- "Nearly sorted array"
- "Online sorting" (elements arrive one by one)
- "Small array"
- "Adaptive sort"

4. When Not to Use It

- When the array is large ($n > 1000$) — $O(n^2)$ is too slow.
- When the array is reverse sorted — worst case $O(n^2)$.
- When you need $O(n \log n)$ performance.
- When comparison cost is very high (it still makes $O(n^2)$ comparisons in worst case).

Simpler alternatives: Built-in `sort()`, Merge Sort, Quick Sort.

5. Core Concepts

Key Element

The element being inserted into the sorted portion. It's temporarily removed from the array.

Shifting vs Swapping

Insertion Sort shifts elements right instead of swapping. This is more efficient than Bubble Sort's swapping approach.

Adaptive

If the array is already sorted (or nearly so), Insertion Sort runs in $O(n)$ time.

Online

Insertion Sort can process elements one at a time as they arrive, making it useful for online sorting scenarios.

Stable

Because we only insert the key after elements strictly greater than it, equal elements maintain their relative order.

6. Step-by-Step Algorithm

```
Input: arr[0..n-1]

1. For i = 1 to n-1:
  a. Set key = arr[i]
  b. Set j = i - 1
  c. While j >= 0 and arr[j] > key:
    arr[j+1] = arr[j]    // shift right
    j--
  d. arr[j+1] = key      // insert key
```

7. Dry Run

Input: [12, 11, 13, 5, 6]

i	key	Sorted Portion	Array Before Insertion	Comparisons	Shifts	Array After Insertion
1	11	[12]	[12, 11, 13, 5, 6]	12 > 11	1	[11, 12, 13, 5, 6]
2	13	[11, 12]	[11, 12, 13, 5, 6]	12 > 13? No	0	[11, 12, 13, 5, 6]
3	5	[11, 12, 13]	[11, 12, 13, 5, 6]	13,12,11 > 5	3	[5, 11, 12, 13, 6]
4	6	[5, 11, 12, 13]	[5, 11, 12, 13, 6]	13,12,11 > 6	3	[5, 6, 11, 12, 13]

Final: [5, 6, 11, 12, 13]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void insertionSort(vector<int>& arr) {
  int n = arr.size();
  for (int i = 1; i < n; ++i) {
    int key = arr[i];
    int j = i - 1;
    // Shift elements greater than key to the right
```

```

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = key;
    }
}

int main() {
    vector<int> arr = {12, 11, 13, 5, 6};
    insertionSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 5 6 11 12 13
    return 0;
}

```

9. Python Implementation

```

def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        # Shift elements greater than key to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

arr = [12, 11, 13, 5, 6]
insertion_sort(arr)
print(arr) # [5, 6, 11, 12, 13]

```

10. Code Explanation

- **Outer loop** ($i = 1$ to $n-1$): The first element is already sorted, so we start from index 1.
- **key = arr[i]**: The element to be inserted into the sorted portion.
- **j = i - 1**: Start from the last element of the sorted portion.
- **While loop**: Shifts elements to the right while they are greater than the key. The condition $j \geq 0$ prevents going out of bounds.
- **arr[j + 1] = key**: Inserts the key in its correct position (the hole created by shifting).

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n)$

Operation	Complexity
Time (Worst)	$O(n^2)$
Time (Average)	$O(n^2)$
Space	$O(1)$
Comparisons	$O(n^2)$
Stable	Yes

12. Common Patterns

Nearly Sorted Array (K-sorted)

- **How to identify:** Each element is at most k positions away from its correct position.
- **Approach:** Insertion Sort runs in $O(n \cdot k)$. For small k , this is efficient.
- **Example:** GFG "Sort a Nearly Sorted Array"

Online Sorting

- **How to identify:** Elements arrive one by one, and you need to maintain a sorted list.
- **Approach:** Use Insertion Sort on each new element.
- **Example:** "Running Median" (use two heaps instead, but Insertion Sort works for small streams)

Sorting Small Subarrays (in Hybrid Algorithms)

- **How to identify:** Used as the base case in Merge Sort/Quick Sort (TimSort does this).
- **Approach:** When subarray size $\leq$ threshold (e.g., 32), use Insertion Sort.

13. Common Mistakes

- **Starting from $i = 0$:** The first element is already sorted; start from $i = 1$.
- **Wrong while condition:** Using `arr[j] >= key` makes the sort unstable (equal elements are moved before each other).
- **Off-by-one in insertion:** `arr[j+1] = key` (not `arr[j]`).
- **Forgetting to decrement j :** Infinite loop.
- **Using $>$ instead of $>=$ for shifting:** The $>$ is correct for stability.

14. Edge Cases

- **Empty array:** No iterations.
- **Single element:** No iterations.
- **Already sorted:** $O(n)$ — each key is already in place, while loop runs once but doesn't shift.
- **Reverse sorted:** $O(n^2)$ — each key must be shifted to the beginning.
- **All equal elements:** $O(n)$ — `arr[j] > key` is false, so no shifts.
- **Large values:** No overflow risk.

15. Variations

Binary Insertion Sort

- Uses binary search to find the insertion position ($O(\log n)$ per element).
- Still $O(n^2)$ due to shifting, but reduces comparisons.
- Useful when comparisons are expensive but swaps are cheap.

Shell Sort

- An extension of Insertion Sort that compares elements far apart first, then reduces the gap.
- Improves time complexity to $O(n \log n)$ or $O(n^{3/2})$ depending on gap sequence.
- Important for placements.

16. Related Algorithms/Data Structures

- **Selection Sort:** Both are $O(n^2)$. Selection Sort minimizes swaps, Insertion Sort is adaptive and stable.
- **Bubble Sort:** Both $O(n^2)$ and stable. Insertion Sort is generally better (fewer moves).
- **Merge Sort:** Uses Insertion Sort for small subarrays in TimSort.
- **Shell Sort:** Direct generalization of Insertion Sort.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Insertion Sort for practice)
- **Insertion Sort** — GFG (implement and test)

Medium

- **Sort a Nearly Sorted Array** — GFG (k-sorted array, $O(n \cdot k)$ solution)
- **Insertion Sort List** — LeetCode 147 (sort a linked list using insertion sort)

Hard

- **Shell Sort** — GFG (implement Shell Sort, an Insertion Sort variant)

18. Interview Explanation

"Insertion Sort builds the sorted array one element at a time. We maintain a sorted prefix, pick the next element (the key), and insert it into its correct position by shifting larger elements to the right. It's adaptive — $O(n)$ for nearly sorted data — and stable. It's excellent for small arrays and is used as the base case in hybrid sorts like TimSort."

19. Revision Notes

- Start from index 1, pick key, shift larger elements right, insert key.
- Adaptive: $O(n)$ for nearly sorted, $O(n^2)$ worst case.
- Stable, in-place, $O(1)$ space.
- Used in TimSort for small subarrays (≤ 32 elements).
- Great for online sorting.

20. Final Cheat Sheet

Property	Value
When to use	Small/nearly sorted arrays, online sorting, TimSort base case
Type	Comparison, in-place, stable, adaptive
Time	$O(n)$ best, $O(n^2)$ worst
Space	$O(1)$
Stable	Yes
Key code idea	<code>while(j >= 0 && arr[j] > key) shift; arr[j+1] = key</code>
Edge cases	Empty, single, sorted, reverse, duplicates

4. MERGE SORT

1. Overview

Merge Sort is a divide-and-conquer sorting algorithm that recursively divides the array into two halves, sorts each half, and then merges the two sorted halves back together.

2. Intuition

Simple explanation: If you have two sorted piles of cards, you can quickly merge them into one sorted pile by repeatedly taking the smaller top card. Merge Sort uses this idea: split the array until each piece is a single element (trivially sorted), then merge them back up.

Analogy: Imagine organizing a tournament. You split participants into smaller groups, each group determines their ranking internally, then you merge the rankings together.

Step-by-step reasoning:

1. If the array has 1 element, it's already sorted.
2. Split the array into two halves.
3. Recursively sort each half.
4. Merge the two sorted halves into one sorted array.
5. The merge step: compare the front elements of each half, pick the smaller one, and repeat.

Why it works: The divide step breaks the problem into smaller subproblems. The merge step combines two sorted arrays in $O(n)$ time. By induction, each half is sorted correctly, so the merge produces a correctly sorted full array.

3. When to Use It

- When you need $O(n \log n)$ guaranteed time (not average like Quick Sort).
- When sorting linked lists (Merge Sort is $O(1)$ space for linked lists).
- When you need a stable sort.

- When external sorting is needed (Merge Sort is the basis for external sorting).
- When data is too large to fit in memory (external Merge Sort).

Trigger phrases:

- "Guaranteed $O(n \log n)$ "
- "Stable sort"
- "Sort linked list"
- "External sort"
- "Count inversions"

4. When Not to Use It

- When memory is constrained (Merge Sort uses $O(n)$ auxiliary space).
- When in-place sorting is required (standard Merge Sort is not in-place).
- When the array is small (Insertion Sort is faster for $n \leq 15-30$).
- When Quick Sort is available and worst-case is not a concern (Quick Sort is faster in practice).

Simpler alternatives: Quick Sort (in-place, faster average), Heap Sort (in-place, $O(n \log n)$ guaranteed), built-in `sort()`.

5. Core Concepts

Divide and Conquer

The problem is divided into smaller subproblems until they are trivial to solve. Then the solutions are combined.

Merge Operation

The heart of the algorithm. Two sorted arrays are merged in $O(n)$ time by repeatedly taking the smaller element from the front of either array.

Recursion Tree

The recursion depth is $O(\log n)$. At each level, $O(n)$ work is done for merging, giving $O(n \log n)$ total.

Auxiliary Space

Standard Merge Sort uses $O(n)$ extra space for the temporary array during merging.

Stable Sorting

Merge Sort is stable because when merging, if equal elements appear, we take from the left array first, preserving relative order.

6. Step-by-Step Algorithm

```
Input: arr[1..r]
```

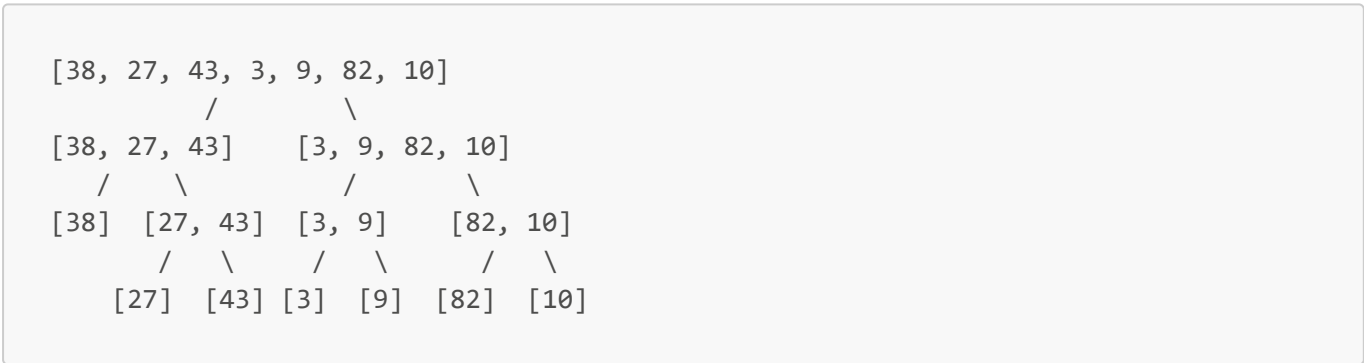
```
mergeSort(arr, l, r):
1. If l >= r, return (base case)
2. mid = (l + r) / 2
3. mergeSort(arr, l, mid)      // sort left half
4. mergeSort(arr, mid+1, r)    // sort right half
5. merge(arr, l, mid, r)      // merge sorted halves

merge(arr, l, mid, r):
1. Create temp array of size r-l+1
2. i = l, j = mid+1, k = 0
3. While i <= mid and j <= r:
    If arr[i] <= arr[j], temp[k++] = arr[i++]
    Else, temp[k++] = arr[j++]
4. Copy remaining elements from left half (if any)
5. Copy remaining elements from right half (if any)
6. Copy temp back to arr[l..r]
```

7. Dry Run

Input: [38, 27, 43, 3, 9, 82, 10]

Divide Phase:



Merge Phase:

```
[38] [27,43] → merge → [27, 38, 43]
[3] [9] → merge → [3, 9]
[82] [10] → merge → [10, 82]
[3, 9] [10, 82] → merge → [3, 9, 10, 82]
[27, 38, 43] [3, 9, 10, 82] → merge → [3, 9, 10, 27, 38, 43, 82]
```

Merge Step Details (last merge):

Left Array	Right Array	Compare	Pick	Result
[27, 38, 43]	[3, 9, 10, 82]	27 vs 3	3	[3]
[27, 38, 43]	[9, 10, 82]	27 vs 9	9	[3, 9]
[27, 38, 43]	[10, 82]	27 vs 10	10	[3, 9, 10]

Left Array	Right Array	Compare	Pick	Result
[27, 38, 43]	[82]	27 vs 82	27	[3, 9, 10, 27]
[38, 43]	[82]	38 vs 82	38	[3, 9, 10, 27, 38]
[43]	[82]	43 vs 82	43	[3, 9, 10, 27, 38, 43]
[]	[82]	-	82	[3, 9, 10, 27, 38, 43, 82]

Final: [3, 9, 10, 27, 38, 43, 82]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Merge two sorted subarrays arr[l..mid] and arr[mid+1..r]
void merge(vector<int>& arr, int l, int mid, int r) {
    int n1 = mid - l + 1;
    int n2 = r - mid;

    // Create temporary arrays
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; ++i) L[i] = arr[l + i];
    for (int i = 0; i < n2; ++i) R[i] = arr[mid + 1 + i];

    // Merge temp arrays back into arr[l..r]
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }

    // Copy remaining elements of L[]
    while (i < n1) arr[k++] = L[i++];
    // Copy remaining elements of R[]
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    merge(arr, l, mid, r);
}

// Wrapper function
void mergeSort(vector<int>& arr) {
```

```

    mergeSort(arr, 0, arr.size() - 1);
}

int main() {
    vector<int> arr = {38, 27, 43, 3, 9, 82, 10};
    mergeSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 3 9 10 27 38 43 82
    return 0;
}

```

9. Python Implementation

```

def merge(arr, l, mid, r):
    """Merge two sorted subarrays arr[l..mid] and arr[mid+1..r]"""
    L = arr[l:mid+1]
    R = arr[mid+1:r+1]

    i = j = 0
    k = l

    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1

    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1

    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

def merge_sort(arr, l, r):
    if l >= r:
        return
    mid = (l + r) // 2
    merge_sort(arr, l, mid)
    merge_sort(arr, mid + 1, r)
    merge(arr, l, mid, r)

# Wrapper
def sort_arr(arr):
    merge_sort(arr, 0, len(arr) - 1)

```

```

    return arr

arr = [38, 27, 43, 3, 9, 82, 10]
sort_arr(arr)
print(arr)  # [3, 9, 10, 27, 38, 43, 82]

```

10. Code Explanation

- **mergeSort(arr, l, r)**: Recursive function. Base case: if $l \geq r$, array has ≤ 1 element.
- **mid = l + (r - l) / 2**: Avoids integer overflow compared to $(l+r)/2$.
- **Two recursive calls**: Sort left half first, then right half.
- **merge() function**: Takes two sorted subarrays and merges them.
- **Temporary arrays L and R**: Copy of the two halves. Extra space is the main cost.
- **Merge loop**: Compares front elements, picks the smaller one.
- **Remaining elements**: Copy anything left over (only one of the two while loops will execute).
- **Copy back**: The merged result is written back into `arr[l..r]`.

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n \log n)$
Time (Worst)	$O(n \log n)$
Time (Average)	$O(n \log n)$
Space	$O(n)$
Stable	Yes

12. Common Patterns

Count Inversions

- **How to identify**: Count pairs (i, j) where $i < j$ and $arr[i] > arr[j]$.
- **Approach**: During merge, when we take from the right array before the left, all remaining elements in the left array are inversions with that element.
- **Example**: LeetCode "Count Inversions" / GFG "Count Inversions"

Sort Linked List

- **How to identify**: You need to sort a linked list.
- **Approach**: Merge Sort is the best choice — $O(n \log n)$ time, $O(1)$ extra space (for linked list, no auxiliary array needed).
- **Example**: LeetCode 148 "Sort List"

External Sorting

- **How to identify**: Data too large to fit in memory.

- **Approach:** Split into chunks, sort each chunk, merge using k-way merge.
- **Example:** "Sort a large file"

K-way Merge (Merge Multiple Sorted Arrays)

- **How to identify:** Merging k sorted arrays into one.
- **Approach:** Use a min-heap for $O(n \log k)$ time.
- **Example:** LeetCode 23 "Merge k Sorted Lists"

13. Common Mistakes

- **Incorrect mid calculation:** $(l+r)/2$ can overflow for large integers. Use $l + (r-l)/2$.
- **Not handling the base case:** `if (l >= r)` should include `l == r` (single element) and `l > r` (empty).
- **Merge loop bounds:** `i < n1` and `j < n2` (not `<=`).
- **Forgetting to copy temp back:** The merged result must be written back to the original array.
- **Using `>` instead of `>=` for stability:** Use `<=` to take from the left array first when equal.
- **Stack overflow:** Merge Sort recursion depth is $O(\log n)$, so stack overflow is rare (unlike Quick Sort).

14. Edge Cases

- **Empty array:** `l > r`, base case returns immediately.
- **Single element:** `l == r`, base case returns immediately.
- **Already sorted:** Still $O(n \log n)$, but merge step just copies.
- **Reverse sorted:** Still $O(n \log n)$.
- **All equal elements:** Still $O(n \log n)$, merge picks from left due to `<=`.

15. Variations

In-Place Merge Sort

- Modifies merge to work in $O(1)$ space using rotations or the "block swap" technique.
- Much more complex; rarely used in practice.
- Not important for placements.

Bottom-Up Merge Sort (Iterative)

- Starts by merging subarrays of size 1, then 2, then 4, etc.
- Avoids recursion, useful for very large arrays.
- Good to know for interviews.

TimSort (Hybrid)

- Combination of Merge Sort and Insertion Sort.
- Used in Python's `sort()`, Java's `Arrays.sort()`, and C++'s `std::stable_sort`.
- We cover this separately in Section 15.

16. Related Algorithms/Data Structures

- **Quick Sort:** Both are divide-and-conquer. Quick Sort is in-place but worst-case $O(n^2)$. Merge Sort is stable and guaranteed $O(n \log n)$ but uses $O(n)$ space.

- **Heap Sort:** In-place, $O(n \log n)$ guaranteed, but not stable. Merge Sort is better for linked lists and external sorting.
- **Divide and Conquer:** Same paradigm as Binary Search, Strassen's Matrix Multiplication, etc.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Merge Sort)
- **Merge Two Sorted Arrays** — LeetCode 88 (practice the merge step)

Medium

- **Sort List** — LeetCode 148 (sort linked list in $O(n \log n)$ time and $O(1)$ space)
- **Count Inversions** — GFG, Codeforces (use merge sort to count inversions)
- **Merge k Sorted Lists** — LeetCode 23 (k-way merge with min-heap)

Hard

- **Median of Two Sorted Arrays** — LeetCode 4 (uses merge-like approach)
- **Reverse Pairs** — LeetCode 493 (similar to inversion count)
- **Skyline Problem** — LeetCode 218 (uses divide and conquer + merge)

18. Interview Explanation

"Merge Sort is a divide-and-conquer algorithm. It divides the array into two halves, recursively sorts each half, and then merges the sorted halves. The merge operation takes $O(n)$ time, and the recursion depth is $O(\log n)$, giving $O(n \log n)$ time in all cases. It's stable and is the basis for external sorting. The main downside is $O(n)$ extra space. I'd use it when guaranteed $O(n \log n)$ is needed, for sorting linked lists, or for count inversions problems."

19. Revision Notes

- Divide: split array into halves.
- Conquer: recursively sort each half.
- Combine: merge two sorted halves.
- $O(n \log n)$ time, $O(n)$ space, stable.
- Merge: compare front elements, pick smaller.
- Base case: $l \geq r$ (0 or 1 element).
- Inversion count: add $mid - i + 1$ when taking from right.

20. Final Cheat Sheet

Property	Value
When to use	Guaranteed $O(n \log n)$, stable sort, linked lists, external sorting, inversion count
Type	Divide-and-conquer, stable
Time	$O(n \log n)$ all cases

Property	Value
Space	$O(n)$
Stable	Yes
Key code idea	<code>mergeSort(left); mergeSort(right); merge(left, right)</code>
Edge cases	Empty, single, sorted, reverse, duplicates

5. QUICK SORT

1. Overview

Quick Sort is a divide-and-conquer algorithm that selects a "pivot" element, partitions the array around it (elements less than pivot on left, greater on right), and then recursively sorts the two partitions.

2. Intuition

Simple explanation: Pick one element as the pivot. Put all smaller elements before it and all larger elements after it. Now the pivot is in its final position. Recursively do the same for the left and right subarrays.

Analogy: Imagine organizing books by height. You pick one book (the pivot). Everyone shorter stands to the left, everyone taller stands to the right. The pivot book is now in its final position. Repeat for the left and right groups.

Step-by-step reasoning:

1. Choose a pivot (e.g., last element, first element, random element, median-of-three).
2. Partition: rearrange the array so that:
 - Elements $\leq$ pivot are on the left.
 - Pivot is in its final position.
 - Elements $\geq$ pivot are on the right.
3. Recursively apply Quick Sort to the left and right subarrays.
4. Base case: subarray of size 0 or 1.

Why it works: After partitioning, the pivot is in its correct sorted position. Recursively sorting the left and right subarrays ensures the entire array is sorted.

3. When to Use It

- When you need fast sorting in practice (Quick Sort is usually faster than Merge Sort and Heap Sort).
- When in-place sorting is required.
- When average-case $O(n \log n)$ is acceptable.
- When you need to find the k-th smallest element (QuickSelect, a variant).
- When the array is random (no worst-case trigger).

Trigger phrases:

- "In-place sort"

- "Fast average sort"
- "Kth smallest/largest element"
- "Partition-based"

4. When Not to Use It

- When $O(n \log n)$ worst-case is required (Quick Sort is $O(n^2)$ worst case).
- When the array is already sorted (with naive pivot selection) — worst case.
- When the array contains many duplicates (Lomuto partition degrades).
- When stability is required (standard Quick Sort is not stable).
- In safety-critical systems where worst-case must be bounded.

Simpler alternatives: Merge Sort (guaranteed $O(n \log n)$, stable), Heap Sort (in-place, guaranteed $O(n \log n)$), `std::sort()` (introsort = Quick Sort + Heap Sort fallback).

5. Core Concepts

Pivot Selection

The choice of pivot dramatically affects performance:

- **First element:** Bad for already sorted arrays ($O(n^2)$).
- **Last element:** Bad for already sorted arrays.
- **Random pivot:** Good average-case, avoids worst-case with high probability.
- **Median-of-three:** Choose median of first, middle, last elements. Good practical choice.

Partitioning

The core operation. Two main schemes:

- **Lomuto partition:** Simple, but degrades with many duplicates.
- **Hoare partition:** More efficient, fewer swaps, better for duplicates.

Lomuto Partition

Uses a single scan. Pivot is the last element. `i` tracks the boundary of elements $\leq$ pivot.

Hoare Partition

Uses two pointers moving toward each other. More efficient (3x fewer swaps on average).

Tail Recursion Optimization

Recursively sort the smaller partition first, then loop for the larger one. This prevents stack overflow for worst-case inputs.

6. Step-by-Step Algorithm (Lomuto Partition)

```
Input: arr[1..r]
```

```

quickSort(arr, l, r):
1. If l >= r, return
2. p = partition(arr, l, r)    // get pivot index
3. quickSort(arr, l, p-1)      // sort left
4. quickSort(arr, p+1, r)      // sort right

partition(arr, l, r):
1. pivot = arr[r]              // choose last element as pivot
2. i = l - 1                   // index of smaller element
3. For j = l to r-1:
    If arr[j] <= pivot:
        i++; swap(arr[i], arr[j])
4. swap(arr[i+1], arr[r])      // place pivot in correct position
5. return i+1                  // return pivot index

```

7. Dry Run

Input: [10, 7, 8, 9, 1, 5] (pivot = last element = 5)

Partition Step:

j	arr[j]	arr[j] ≤ 5?	arr before swap	i	arr after swap
0	10	No	[10, 7, 8, 9, 1, 5]	-1	[10, 7, 8, 9, 1, 5]
1	7	No	[10, 7, 8, 9, 1, 5]	-1	[10, 7, 8, 9, 1, 5]
2	8	No	[10, 7, 8, 9, 1, 5]	-1	[10, 7, 8, 9, 1, 5]
3	9	No	[10, 7, 8, 9, 1, 5]	-1	[10, 7, 8, 9, 1, 5]
4	1	Yes	[10, 7, 8, 9, 1, 5]	0	[1, 7, 8, 9, 10, 5]

After loop: `swap(arr[0+1], arr[5])` → [1, 5, 8, 9, 10, 7]

Pivot index = 1.

Recursive calls:

- Left: `quickSort([1], 0, 0)` → base case.
- Right: `quickSort([8, 9, 10, 7], 2, 5)` → partition with pivot=7 → [1, 5, 7, 9, 10, 8], pivot index = 2.
 - Left: `quickSort([], 2, 1)` → base case.
 - Right: `quickSort([9, 10, 8], 3, 5)` → partition with pivot=8 → [1, 5, 7, 8, 10, 9], pivot index = 3.
 - Left: `quickSort([], 3, 2)` → base case.
 - Right: `quickSort([10, 9], 4, 5)` → partition with pivot=9 → [1, 5, 7, 8, 9, 10], pivot index = 4.
 - Left: base case.
 - Right: base case.

Final: [1, 5, 7, 8, 9, 10]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Lomuto partition scheme
int partition(vector<int>& arr, int l, int r) {
    int pivot = arr[r]; // choose last element as pivot
    int i = l - 1;      // index of smaller element

    for (int j = l; j < r; ++j) {
        if (arr[j] <= pivot) {
            ++i;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[r]); // place pivot in correct position
    return i + 1;
}

// Hoare partition scheme (more efficient)
int partitionHoare(vector<int>& arr, int l, int r) {
    int pivot = arr[l + (r - l) / 2]; // middle element as pivot
    int i = l - 1, j = r + 1;

    while (true) {
        do { ++i; } while (arr[i] < pivot);
        do { --j; } while (arr[j] > pivot);
        if (i >= j) return j;
        swap(arr[i], arr[j]);
    }
}

void quickSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;

    // Use random pivot to avoid worst-case on sorted input
    int randomIdx = l + rand() % (r - l + 1);
    swap(arr[randomIdx], arr[r]);

    int p = partition(arr, l, r);
    quickSort(arr, l, p - 1);
    quickSort(arr, p + 1, r);
}

// Wrapper
void quickSort(vector<int>& arr) {
    srand(time(0));
    quickSort(arr, 0, arr.size() - 1);
}

// Tail-recursion optimized version (prevents stack overflow)
```

```

void quickSortOptimized(vector<int>& arr, int l, int r) {
    while (l < r) {
        int p = partition(arr, l, r);
        // Recursively sort the smaller partition
        if (p - l < r - p) {
            quickSortOptimized(arr, l, p - 1);
            l = p + 1;
        } else {
            quickSortOptimized(arr, p + 1, r);
            r = p - 1;
        }
    }
}

int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    quickSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 1 5 7 8 9 10
    return 0;
}

```

9. Python Implementation

```

import random

def partition(arr, l, r):
    """Lomuto partition with random pivot"""
    random_idx = random.randint(l, r)
    arr[random_idx], arr[r] = arr[r], arr[random_idx]

    pivot = arr[r]
    i = l - 1

    for j in range(l, r):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[r] = arr[r], arr[i + 1]
    return i + 1

def quick_sort(arr, l, r):
    if l >= r:
        return
    p = partition(arr, l, r)
    quick_sort(arr, l, p - 1)
    quick_sort(arr, p + 1, r)

def sort_arr(arr):
    quick_sort(arr, 0, len(arr) - 1)

```

```

return arr

arr = [10, 7, 8, 9, 1, 5]
sort_arr(arr)
print(arr) # [1, 5, 7, 8, 9, 10]

```

10. Code Explanation

- **partition():** Rearranges the array so that elements $\leq$ pivot come before, and elements $\geq$ pivot come after.
- **i = l - 1:** Tracks the boundary of elements $\leq$ pivot. Everything up to index **i** is $\leq$ pivot.
- **Loop j = l to r-1:** Scans the array. If **arr[j] <= pivot**, increment **i** and swap.
- **Final swap:** Places the pivot at **i+1**, which is its correct position.
- **quickSort():** Recursively sorts left and right partitions.
- **Random pivot:** Swapping a random element with **arr[r]** prevents worst-case on sorted input.
- **Tail recursion optimization:** The optimized version uses a loop for the larger partition to avoid deep recursion.

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n \log n)$
Time (Worst)	$O(n^2)$
Time (Average)	$O(n \log n)$
Space (Worst)	$O(n)$
Space (Avg)	$O(\log n)$
Stable	No

12. Common Patterns

QuickSelect (Kth Smallest/Largest Element)

- **How to identify:** Find the k-th smallest or largest element without sorting the whole array.
- **Approach:** Use partition. If pivot index == k-1, return pivot. If pivot index > k-1, recurse on left. Else recurse on right.
- **Complexity:** $O(n)$ average, $O(n^2)$ worst.
- **Example:** LeetCode 215 "Kth Largest Element in an Array"

Dutch National Flag (3-Way Partition)

- **How to identify:** Array with three distinct values (e.g., sort 0s, 1s, 2s).
- **Approach:** Use three-way partition with three pointers: low, mid, high.
- **Example:** LeetCode 75 "Sort Colors"

Partition in Quick Sort

- **How to identify:** Problems that ask to rearrange elements around a pivot.
- **Approach:** Use Lomuto or Hoare partition.
- **Example:** "Segregate 0s and 1s", "Move all negative numbers to beginning"

13. Common Mistakes

- **Stack overflow:** Worst-case (sorted array with naive pivot) leads to $O(n)$ recursion depth. Fix with random pivot or tail recursion optimization.
- **Infinite recursion:** Incorrect base case condition. Use `if (l >= r) return;`.
- **Off-by-one in partition:** Lomuto partition returns `i+1`, not `i`.
- **Forgetting to randomize pivot:** Makes the algorithm vulnerable to $O(n^2)$ worst-case on sorted input.
- **Using `>` instead of `>=` for duplicates:** With Lomuto partition, using `>` causes infinite recursion for arrays with equal elements.
- **Not stable:** Quick Sort is not stable. Don't use it when stability matters.

14. Edge Cases

- **Empty array:** Base case, no recursion.
- **Single element:** Base case, no recursion.
- **Already sorted:** $O(n^2)$ with naive pivot, $O(n \log n)$ with random pivot.
- **Reverse sorted:** Same as already sorted.
- **All equal elements:** Lomuto partition gives $O(n^2)$ because all elements are $\leq$ pivot. Use Hoare partition or 3-way partition.
- **Large values:** No overflow risk (only comparisons and swaps).

15. Variations

3-Way Quick Sort (Dutch National Flag)

- Partitions into three regions: $<$ pivot, $=$ pivot, $>$ pivot.
- Handles duplicates efficiently.
- $O(n \log n)$ even with many duplicates.

QuickSelect

- Finds the k -th smallest element without full sorting.
- $O(n)$ average, $O(n^2)$ worst.
- Very important for placements.

Introsort

- Hybrid: Quick Sort + Heap Sort + Insertion Sort.
- Uses Quick Sort, falls back to Heap Sort if recursion depth exceeds $\log n$.
- Used in `std::sort()` in C++.
- Important to know conceptually.

Dual-Pivot Quick Sort

- Uses two pivots, partitions into three regions.

- Used in Java's `Arrays.sort()`.
- Slightly faster in practice.

16. Related Algorithms/Data Structures

- **Merge Sort:** Both divide-and-conquer. Merge Sort is stable and guaranteed $O(n \log n)$ but uses $O(n)$ space. Quick Sort is in-place but $O(n^2)$ worst case.
- **Heap Sort:** In-place, $O(n \log n)$ guaranteed. Quick Sort is faster in practice.
- **QuickSelect:** A Quick Sort variant for finding k-th order statistics.
- **Binary Search:** Both use divide-and-conquer, but Binary Search is $O(\log n)$ and works on sorted data.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Quick Sort)
- **Sort Colors** — LeetCode 75 (3-way partition, Dutch National Flag)

Medium

- **Kth Largest Element in an Array** — LeetCode 215 (QuickSelect)
- **Top K Frequent Elements** — LeetCode 347 (QuickSelect on frequency array)
- **K Closest Points to Origin** — LeetCode 973 (QuickSelect)

Hard

- **Wiggle Sort II** — LeetCode 324 (uses QuickSelect + partition)
- **Median of Two Sorted Arrays** — LeetCode 4 (uses QuickSelect concept)

18. Interview Explanation

"Quick Sort is a divide-and-conquer algorithm that picks a pivot, partitions the array around it so that smaller elements are on the left and larger on the right, and then recursively sorts the two partitions. The partitioning step is the key — it's an in-place operation that places the pivot in its final sorted position. With random pivot selection, Quick Sort runs in $O(n \log n)$ average time and $O(\log n)$ space. The worst-case is $O(n^2)$, but randomization makes this vanishingly unlikely. I'd use it when I need a fast in-place sort."

19. Revision Notes

- Pick pivot, partition, recurse on left and right.
- Always randomize pivot to avoid $O(n^2)$.
- Lomuto partition: `i` tracks boundary of $\leq$ pivot.
- Hoare partition: two pointers, better for duplicates.
- $O(n \log n)$ average, $O(n^2)$ worst, $O(\log n)$ space average.
- Not stable.
- QuickSelect for k-th smallest: $O(n)$ average.

20. Final Cheat Sheet

Property	Value
When to use	Fast in-place sort, QuickSelect
Type	Divide-and-conquer, in-place
Time	$O(n \log n)$ avg, $O(n^2)$ worst
Space	$O(\log n)$ avg, $O(n)$ worst
Stable	No
Key code idea	<code>partition()</code> → pivot in correct position
Edge cases	Empty, single, sorted, reverse, duplicates, all equal

6. COUNTING SORT

1. Overview

Counting Sort is a non-comparison-based sorting algorithm that sorts integers by counting the occurrences of each distinct value and using that count to determine the positions of elements in the sorted output.

2. Intuition

Simple explanation: Instead of comparing elements, count how many times each value appears. Then use the cumulative counts to place each element in its correct position.

Analogy: Imagine sorting exam scores (0–100). You have 100 buckets. You go through the papers and put each one in the bucket for its score. Then you collect the buckets in order — all 0s first, then all 1s, etc.

Step-by-step reasoning:

1. Find the range of input values (min and max).
2. Create a count array of size $\text{max} - \text{min} + 1$.
3. Count occurrences of each value.
4. Transform counts into cumulative counts (prefix sums).
5. Iterate through the input from right to left, place each element in its correct position in the output array using the cumulative count.
6. Copy the output array back.

Why it works: The cumulative count tells us, for each value, how many elements are $\leq$ that value. This gives the exact position (1-indexed) of each element in the sorted array.

3. When to Use It

- When the range of input values is small ($\leq 10^6$).
- When the input contains integers (or discrete values that can be mapped to integers).
- When you need $O(n)$ time sorting.
- When the input size is large but the range is small.
- As a subroutine for Radix Sort.

Trigger phrases:

- "Linear time sort"
- "Small range of values"
- "Integer sorting"
- "Sort by frequency"
- "Non-comparison sort"

4. When Not to Use It

- When the range of values is very large (e.g., 10^9) — the count array would be too large.
- When sorting floating-point numbers (unless they are discrete and can be mapped).
- When sorting strings or custom objects (unless you can map them to integers).
- When the input is small (comparison sorts are simpler and fast enough).
- When memory is limited and the range is large.

Simpler alternatives: `sort()` from the standard library, Radix Sort (for large range but limited digits), Bucket Sort (for floating-point).

5. Core Concepts

Range (k)

The difference between the maximum and minimum value plus 1. Determines the size of the count array.

Count Array

An array of size `k` where `count[i]` stores the frequency of value `min + i`.

Cumulative Count (Prefix Sum)

After computing `count`, we transform it into a prefix sum. `count[i]` now stores the number of elements $\leq \text{min} + i$.

Stable Placement

By iterating the input from right to left and using the cumulative count, the sort is stable. Each element is placed at `output[count[value] - 1]`, and then `count[value]` is decremented.

6. Step-by-Step Algorithm

Input: `arr[0..n-1]`

1. Find min and max values in `arr`.
2. `range = max - min + 1`
3. Create count array of size `range`, initialized to 0.
4. For each element `x` in `arr`:
 `count[x - min]++`
5. Transform count to prefix sums:
 For `i = 1` to `range-1`:

```

        count[i] += count[i-1]
6. Create output array of size n.
7. For i = n-1 down to 0:
    val = arr[i]
    pos = count[val - min] - 1
    output[pos] = val
    count[val - min]--
8. Copy output back to arr.

```

7. Dry Run

Input: [4, 2, 2, 8, 3, 3, 1]

- min = 1, max = 8, range = 8

Step 1: Count frequencies

Value	1	2	3	4	5	6	7	8
Count	1	2	2	1	0	0	0	1

Step 2: Cumulative counts

Value	1	2	3	4	5	6	7	8
Count	1	3	5	6	6	6	6	7

Step 3: Build output (right to left)

i	arr[i]	pos = count[val-1]-1	output (before decrement)	after decrement
6	1	1-1 = 0	[1, _ _ _ _ _]	count[0]=0
5	3	5-1 = 4	[1, _ _ _ 3, _]	count[2]=4
4	3	4-1 = 3	[1, _ _ 3, 3, _]	count[2]=3
3	8	7-1 = 6	[1, _ _ 3, 3, _ 8]	count[7]=6
2	2	3-1 = 2	[1, _ 2, 3, 3, _ 8]	count[1]=2
1	2	2-1 = 1	[1, 2, 2, 3, 3, _ 8]	count[1]=1
0	4	6-1 = 5	[1, 2, 2, 3, 3, 4, 8]	count[3]=5

Final: [1, 2, 2, 3, 3, 4, 8]

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

void countingSort(vector<int>& arr) {

```

```

if (arr.empty()) return;

// Find range of values
int minVal = *min_element(arr.begin(), arr.end());
int maxVal = *max_element(arr.begin(), arr.end());
int range = maxVal - minVal + 1;

// Count frequencies
vector<int> count(range, 0);
for (int x : arr) {
    count[x - minVal]++;
}

// Transform to prefix sums (cumulative count)
for (int i = 1; i < range; ++i) {
    count[i] += count[i - 1];
}

// Build output array (stable sort by iterating from right to left)
vector<int> output(arr.size());
for (int i = arr.size() - 1; i >= 0; --i) {
    int val = arr[i];
    int pos = count[val - minVal] - 1;
    output[pos] = val;
    count[val - minVal]--;
}

// Copy back
arr = output;
}

int main() {
    vector<int> arr = {4, 2, 2, 8, 3, 3, 1};
    countingSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 1 2 2 3 3 4 8
    return 0;
}

```

9. Python Implementation

```

def counting_sort(arr):
    if not arr:
        return arr

    min_val = min(arr)
    max_val = max(arr)
    range_val = max_val - min_val + 1

    # Count frequencies
    count = [0] * range_val

```

```

for x in arr:
    count[x - min_val] += 1

# Transform to prefix sums
for i in range(1, range_val):
    count[i] += count[i - 1]

# Build output array (stable)
output = [0] * len(arr)
for i in range(len(arr) - 1, -1, -1):
    val = arr[i]
    pos = count[val - min_val] - 1
    output[pos] = val
    count[val - min_val] -= 1

return output

arr = [4, 2, 2, 8, 3, 3, 1]
print(counting_sort(arr)) # [1, 2, 2, 3, 3, 4, 8]

```

10. Code Explanation

- **Find min/max:** Determines the range of values. This is necessary because we need a count array of exactly `range` size.
- **Count frequencies:** `count[x - minVal]++` maps each value to an index in the count array.
- **Prefix sum:** `count[i] += count[i-1]` transforms the count array so that `count[i]` = number of elements $\leq \text{minVal} + i$.
- **Right-to-left iteration:** This ensures stability. When we encounter equal elements, the one that appears later in the original array is placed later in the output (because we decrement `count[val - minVal]` after placing).
- **Position calculation:** `pos = count[val - minVal] - 1` gives the 0-indexed position of the current element.

11. Complexity Analysis

Operation	Complexity
Time	$O(n + k)$
Space	$O(k)$
Stable	Yes

Note: When $k = O(n)$, Counting Sort runs in $O(n)$ time. When $k \gg n$, the algorithm is inefficient.

12. Common Patterns

Sorting by Frequency

- **How to identify:** Sort elements by their frequency (not by value).

- **Approach:** Count frequencies, then sort the unique values by frequency using Counting Sort (if frequency range is small).
- **Example:** LeetCode 451 "Sort Characters By Frequency"

Sort Characters by ASCII

- **How to identify:** Sorting a string of characters.
- **Approach:** Counting Sort with range = 256 (ASCII).
- **Example:** "Sort a string of lowercase letters" (range = 26)

Find Missing Number / Duplicate

- **How to identify:** Problems where you need to find missing or duplicate numbers in a range.
- **Approach:** Use Counting Sort's frequency array.
- **Example:** LeetCode 268 "Missing Number" (use XOR, but Counting Sort works too)

13. Common Mistakes

- **Negative values:** Forgetting to handle negative values by subtracting `minVal`. Always use `x - minVal` as the index.
- **Huge range with sparse data:** Using Counting Sort when max-min is large (e.g., 10^9) but `n` is small — this allocates a huge count array.
- **Off-by-one in prefix sum:** After prefix sum, `count[val - minVal]` gives the count of elements $\leq$ `val`, which is the position (1-indexed). So `pos = count[...] - 1` is the 0-indexed position.
- **Not iterating from right to left:** This breaks stability. The sort still works, but it's no longer stable.
- **Forgetting to copy back:** The output array must be copied back to the original array.

14. Edge Cases

- **Empty array:** Return immediately.
- **Single element:** Works correctly.
- **All same values:** `count[0] = n`, prefix sum = `n`, all elements placed in `output[n-1..0]` correctly.
- **Negative values:** Handled by subtracting `minVal`.
- **Large range:** If `maxVal - minVal` is large (e.g., $> 10^7$), the count array is too large. Use Radix Sort or comparison sort.

15. Variations

Counting Sort for Negative Numbers

- Standard Counting Sort handles negatives by shifting by `minVal`.
- This is the default approach in the implementation above.

Counting Sort for Objects

- Can be used if objects have integer keys.
- The key is extracted, and the object is placed in the output array using the counting sort logic.

16. Related Algorithms/Data Structures

- **Radix Sort:** Uses Counting Sort as a subroutine for each digit.
- **Bucket Sort:** Also non-comparison, but uses buckets and sorts within each bucket.
- **Pigeonhole Sort:** Similar to Counting Sort but uses a different placement strategy.
- **Frequency Array / Hash Map:** Counting Sort is essentially a frequency array that is used for sorting.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (use Counting Sort for practice if range is small)
- **Sort Colors** — LeetCode 75 (Counting Sort with range = 3)

Medium

- **Sort Characters By Frequency** — LeetCode 451 (frequency + counting sort)
- **Top K Frequent Elements** — LeetCode 347 (bucket sort variant)

Hard

- **Maximum Gap** — LeetCode 164 (uses bucket sort / counting sort ideas)
- **First Missing Positive** — LeetCode 41 (uses counting sort like placement)

18. Interview Explanation

"Counting Sort is a non-comparison-based sort that works in $O(n + k)$ time, where k is the range of values. It counts the frequency of each distinct value, computes cumulative counts, and uses those to place each element in its correct position. It's stable, requires $O(k)$ extra space, and is efficient when the range of values is not significantly larger than the input size. It's used as a subroutine in Radix Sort."

19. Revision Notes

- Non-comparison sort: $O(n + k)$ time.
- Requires integer values (or discrete values mapped to integers).
- Stable if iterated from right to left.
- Range $k = \text{max} - \text{min} + 1$. Need $O(k)$ space.
- Step: count $\rightarrow$ prefix sum $\rightarrow$ place right to left.
- Not suitable for large ranges.

20. Final Cheat Sheet

Property	Value
When to use	Small range of integers, $O(n)$ needed
Type	Non-comparison, stable
Time	$O(n + k)$
Space	$O(k)$
Stable	Yes (if right-to-left)

Property	Value
Key code idea	<code>count[val-min]++</code> → prefix sum → <code>output[count[val-min]-1] = val</code>
Edge cases	Empty, single, all same, negative values, large range

7. CUSTOM COMPARATOR

1. Overview

A custom comparator is a function or functor that defines a custom ordering for sorting. Instead of using the default `<` operator, you provide a comparator that returns `true` if the first argument should appear before the second in the sorted order.

2. Intuition

Simple explanation: The default sort orders numbers from smallest to largest. But what if you want to sort by last digit, or by the sum of digits, or by a custom priority? A custom comparator lets you define your own "less than" relationship.

Analogy: In a contest, you might want to sort participants by score (descending), then by name (alphabetical). The default sort doesn't know this. You write a comparator that says: "Person A comes before Person B if A's score > B's score, or if scores are equal and A's name < B's name."

Step-by-step reasoning:

1. Define a function `cmp(a, b)` that returns `true` if `a` should come before `b`.
2. Pass this function to the sort function.
3. The sort algorithm uses `cmp` instead of `<` for comparisons.

Why it works: Most sorting algorithms only need to compare two elements at a time. By replacing `<` with your custom comparator, you can define any ordering.

3. When to Use It

- When sorting custom objects (structs, pairs, tuples).
- When sorting by a key that is not the default (e.g., sort by absolute value, by last digit, by frequency).
- When sorting in descending order.
- When sorting by multiple criteria (primary key, secondary key, etc.).
- When sorting intervals, points, or complex data structures.

Trigger phrases:

- "Sort by ..."
- "Custom ordering"
- "Sort pairs"
- "Multiple criteria"
- "Comparator function"

4. When Not to Use It

- When the default `<` operator already gives the required ordering.
- When you can transform the data and use the default sort (e.g., sort by absolute value: store the absolute values and sort normally).
- When the comparator is not a strict weak ordering (see below).

Simpler alternatives: Default sort, transform-and-sort, `std::tuple` lexicographic comparison.

5. Core Concepts

Strict Weak Ordering

A comparator must satisfy:

- **Irreflexive:** `cmp(a, a)` is always `false`.
- **Antisymmetric:** If `cmp(a, b)` is `true`, then `cmp(b, a)` is `false`.
- **Transitive:** If `cmp(a, b)` and `cmp(b, c)` are `true`, then `cmp(a, c)` is `true`.
- **Transitivity of equivalence:** If `cmp(a, b)` and `cmp(b, a)` are both `false` (a and b are equivalent), and `cmp(b, c)` and `cmp(c, b)` are both `false`, then `cmp(a, c)` and `cmp(c, a)` must be `false`.

A comparator that violates these rules leads to undefined behavior.

Lambda Functions

In modern C++, lambdas are the most common way to write comparators:

```
sort(arr.begin(), arr.end(), [](int a, int b) {
    return a > b; // descending order
});
```

Functors (Function Objects)

A struct with `operator()` overloaded:

```
struct Compare {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) {
        return a.second < b.second;
    }
};
sort(arr.begin(), arr.end(), Compare());
```

Comparator for Priority Queue / Set

For `priority_queue`, `set`, `map`, `multiset`, etc., the comparator is passed as a template argument:

```
priority_queue<int, vector<int>, greater<int>> pq; // min-heap
```

6. Step-by-Step Algorithm

Not an algorithm per se – a concept used with sorting functions.

1. Decide the criteria for ordering.
2. Write a function that takes two elements and returns true if the first should come before the second.
3. Pass the function to the sorting function.
4. The sorting algorithm applies the comparator throughout.

7. Dry Run

Input: `arr = [3, 1, 4, 1, 5, 9, 2, 6]` **Comparator:** `cmp(a, b) = (a % 10) < (b % 10)` (sort by last digit)

Element	Last Digit
3	3
1	1
4	4
1	1
5	5
9	9
2	2
6	6

Sorted by Last Digit: `[1, 1, 2, 3, 4, 5, 6, 9]`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Example 1: Sort by last digit (ascending)
void sortByLastDigit(vector<int>& arr) {
    sort(arr.begin(), arr.end(), [](int a, int b) {
        return (a % 10) < (b % 10);
    });
}
```

```
// Example 2: Sort pairs by second element, then by first
void sortPairs(vector<pair<int, int>>& arr) {
    sort(arr.begin(), arr.end(), [](auto& a, auto& b) {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first;
    });
}

// Example 3: Sort strings by length, then lexicographically
void sortStrings(vector<string>& arr) {
    sort(arr.begin(), arr.end(), [](const string& a, const string& b) {
        if (a.size() != b.size()) return a.size() < b.size();
        return a < b;
    });
}

// Example 4: Custom struct comparator
struct Person {
    string name;
    int age;
    double salary;
};

void sortPersons(vector<Person>& people) {
    sort(people.begin(), people.end(), [](const Person& a, const Person& b) {
        // Sort by age descending, then salary ascending, then name
        if (a.age != b.age) return a.age > b.age;
        if (a.salary != b.salary) return a.salary < b.salary;
        return a.name < b.name;
    });
}

// Example 5: Comparator for priority_queue (min-heap based on frequency)
void priorityQueueExample() {
    // Min-heap: smallest frequency at top
    auto cmp = [](pair<int, int> a, pair<int, int> b) {
        return a.second > b.second; // Note: > for min-heap!
    };
    priority_queue<pair<int, int>, vector<pair<int, int>>, decltype(cmp)> pq(cmp);
}

// Example 6: Sorting with indices (decorate-sort-undecorate)
vector<int> sortWithIndices(const vector<int>& arr) {
    int n = arr.size();
    vector<pair<int, int>> indexed(n);
    for (int i = 0; i < n; ++i) indexed[i] = {arr[i], i};

    sort(indexed.begin(), indexed.end(), [](auto& a, auto& b) {
        if (a.first != b.first) return a.first < b.first;
        return a.second < b.second; // stable by index
    });

    vector<int> indices(n);
    for (int i = 0; i < n; ++i) indices[i] = indexed[i].second;
}
```

```

        return indices;
    }

    int main() {
        vector<int> arr = {3, 1, 4, 1, 5, 9, 2, 6};
        sortByLastDigit(arr);
        for (int x : arr) cout << x << " ";
        // Output: 1 1 2 3 4 5 6 9
        return 0;
    }

```

9. Python Implementation

```

# Example 1: Sort by last digit
arr = [3, 1, 4, 1, 5, 9, 2, 6]
arr.sort(key=lambda x: x % 10)
print(arr) # [1, 1, 2, 3, 4, 5, 6, 9]

# Example 2: Sort pairs by second element, then first
pairs = [(1, 3), (2, 2), (3, 1), (4, 2)]
pairs.sort(key=lambda x: (x[1], x[0]))
print(pairs) # [(3, 1), (2, 2), (4, 2), (1, 3)]

# Example 3: Sort strings by length, then lexicographically
strings = ["apple", "banana", "cat", "dog", "elephant"]
strings.sort(key=lambda x: (len(x), x))
print(strings) # ['cat', 'dog', 'apple', 'banana', 'elephant']

# Example 4: Custom class with comparator
from functools import cmp_to_key

class Person:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def __repr__(self):
        return f"{self.name}({self.age}, {self.salary})"

def person_cmp(a, b):
    if a.age != b.age:
        return b.age - a.age # descending
    if a.salary != b.salary:
        return -1 if a.salary < b.salary else 1 # ascending
    if a.name != b.name:
        return -1 if a.name < b.name else 1
    return 0

people = [
    Person("Alice", 30, 50000),

```

```

    Person("Bob", 25, 60000),
    Person("Charlie", 30, 50000),
    Person("David", 30, 40000),
]
people.sort(key=cmp_to_key(person_cmp))
print(people)
# [Charlie(30, 50000), Alice(30, 50000), David(30, 40000), Bob(25, 60000)]

# Example 5: Sort with indices (decorate-sort-undecorate)
arr = [4, 2, 2, 8, 3, 3, 1]
indexed = [(val, idx) for idx, val in enumerate(arr)]
indexed.sort(key=lambda x: (x[0], x[1]))
indices = [idx for _, idx in indexed]
print(indices) # [6, 1, 2, 4, 5, 0, 3]

```

10. Code Explanation

- **Lambda in C++:** `[](int a, int b) { return a % 10 < b % 10; }` defines an anonymous comparator.
- **Multiple criteria:** Check primary key first. If equal, check secondary key. Chain until all criteria are checked.
- **auto& parameters:** Use `const auto&` or `const T&` to avoid copying.
- **Priority queue comparator:** Note the `>` for min-heap. The PQ uses `comp(a, b)` to check if `a` should go below `b`.
- **cmp_to_key in Python:** Python's `sort` only takes `key`, not a comparator. Use `cmp_to_key` from `functools` for compatibility.
- **key in Python:** Simpler than a comparator. `key` is a function that extracts a value, and elements are sorted by that value.

11. Complexity Analysis

Operation	Complexity
Sorting with custom comparator	$O(n \log n)$
Comparator call overhead	$O(\log n)$ per comparison
Space	$O(\log n)$

12. Common Patterns

Sort by Multiple Keys

- **How to identify:** Problems with multiple sorting criteria (e.g., sort by score descending, then by name ascending).
- **Approach:** Chain conditions in the comparator.
- **Example:** "Sort employees by salary, then by name"

Sort by Transformed Value (Key Function)

- **How to identify:** Sorting by a computed value (e.g., sort by absolute value, by distance from origin).
- **Approach:** Use `key` in Python, or compute the value in the comparator in C++.
- **Example:** LeetCode 973 "K Closest Points to Origin"

Sort by Frequency

- **How to identify:** Sort elements by how often they appear, not by their value.
- **Approach:** Count frequencies first, then sort by frequency.
- **Example:** LeetCode 451 "Sort Characters By Frequency"

Sort with Indices

- **How to identify:** Need to track the original positions after sorting.
- **Approach:** Store (value, index) pairs, sort with custom comparator.
- **Example:** "Find the rank of each element"

13. Common Mistakes

- **Violating strict weak ordering:** If `cmp(a, b)` and `cmp(b, a)` are both `true`, the behavior is undefined.
- **Using `<=` instead of `<`:** `cmp(a, a)` must be `false`. `return a <= b` is wrong because `cmp(a, a)` returns `true`.
- **Comparator for priority queue is backwards:** `priority_queue<T, vector<T>, greater<T>>` gives a min-heap. Custom comparator uses `>` for min-heap.
- **Not capturing needed variables:** Lambda must capture external variables. Use `[&]` or `[=]` or specific captures.
- **Expensive comparator:** If the comparator is $O(k)$ and called $O(n \log n)$ times, total time is $O(k \cdot n \log n)$. Precompute the key.
- **Incorrect `const` correctness:** Comparator should take `const` references to avoid copying.

14. Edge Cases

- **All equal elements:** Comparator returns `false` for both directions. This is fine.
- **Empty range:** Sort with custom comparator on an empty range is a no-op.
- **Single element:** Single element is always sorted.
- **Large objects:** Use `const T&` in comparator to avoid copying large objects.
- **Floating point:** Be careful with floating point comparisons. Use `abs(a - b) < eps` for equality.

15. Variations

Python's `key` vs `cmp`

- `key` is simpler and more efficient (called once per element).
- `cmp` is more flexible (can compare two elements directly).
- In Python 3, `sort()` no longer accepts `cmp` directly. Use `cmp_to_key`.

`nth_element` with Custom Comparator

- `std::nth_element` can take a custom comparator.

- Used for partial sorting (e.g., find top K elements).

`stable_sort` with Custom Comparator

- `std::stable_sort` preserves the relative order of equivalent elements.
- Takes a custom comparator like `sort`.

16. Related Algorithms/Data Structures

- **Sorting Algorithms:** All sorting algorithms can take a custom comparator.
- **Priority Queue:** Uses a comparator for its ordering.
- **Set / Map:** Use a comparator for ordering of keys.
- **Binary Search:** `lower_bound/upper_bound` can take a custom comparator.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (practice with custom comparators)
- **Sort Colors** — LeetCode 75 (can use custom comparator)

Medium

- **Sort Characters By Frequency** — LeetCode 451 (frequency-based comparator)
- **K Closest Points to Origin** — LeetCode 973 (distance-based comparator)
- **Largest Number** — LeetCode 179 (custom comparator for concatenation)

Hard

- **Minimum Interval to Include Each Query** — LeetCode 1851 (interval sorting with custom comparator)
- **Maximum Profit in Job Scheduling** — LeetCode 1235 (sort by end time)

18. Interview Explanation

"A custom comparator defines a custom ordering for sorting. In C++, you pass a lambda or function to `sort()` that returns `true` if the first argument should come before the second. The comparator must satisfy strict weak ordering — it must be irreflexive, antisymmetric, and transitive. In Python, you use the `key` parameter to extract a sort key, or `cmp_to_key` for more complex comparisons. I use custom comparators whenever I need to sort by non-default criteria or sort custom objects."

19. Revision Notes

- Lambda: `[](const T& a, const T& b) { return /* a before b */; }`
- Strict weak ordering: `cmp(a,a)=false`, transitive, antisymmetric.
- Python `key` is simpler: `sort(key=lambda x: x[1])`.
- Priority queue: `>` for min-heap in custom comparator.
- Multiple criteria: chain conditions.
- Precompute expensive keys.

20. Final Cheat Sheet

Property	Value
When to use	Custom ordering, multi-key sort, objects
How (C++)	<code>sort(v.begin(), v.end(), [](T a, T b){ return ...; })</code>
How (Python)	<code>sort(key=lambda x: ...)</code> or <code>sort(key=cmp_to_key(fn))</code>
Requirements	Strict weak ordering
Key code idea	Return <code>true</code> if <code>a</code> should come before <code>b</code>
Edge cases	All equal, empty, large objects

8. SORTING INTERVALS

1. Overview

Sorting intervals is a specific application of sorting where we sort intervals (pairs of start and end times) by their start time, end time, or length. It's a fundamental preprocessing step for interval-based problems.

2. Intuition

Simple explanation: An interval is a range `[start, end]`. Sorting intervals by start time helps us find overlaps, merge intervals, or find gaps. Sorting by end time helps with scheduling problems.

Analogy: Imagine a calendar with meetings. Sorting by start time tells you which meeting starts first. Sorting by end time tells you which meeting ends first.

Step-by-step reasoning:

1. Intervals are usually represented as pairs `(start, end)`.
2. Sorting by start: `sort by start, then by end` — useful for merging.
3. Sorting by end: `sort by end, then by start` — useful for scheduling.
4. Sorting by length: `sort by (end - start)` — useful for finding smallest/largest intervals.

Why it works: Many interval problems become trivial once intervals are sorted by a specific key. For example, to merge overlapping intervals, sorting by start time lets us process them in order and check overlaps in $O(n)$ time.

3. When to Use It

- When merging overlapping intervals.
- When finding non-overlapping intervals (scheduling).
- When finding the minimum number of resources (rooms, platforms) needed.
- When finding gaps in intervals.
- When checking if a point is covered by any interval.
- When inserting a new interval into a sorted list.

Trigger phrases:

- "Merge intervals"
- "Overlapping intervals"
- "Non-overlapping intervals"
- "Minimum number of ..."
- "Meeting rooms"
- "Interval scheduling"

4. When Not to Use It

- When intervals are already sorted (rare, but possible).
- When the problem doesn't require ordering (e.g., just counting points covered).
- When range-based data structures (segment tree, difference array) are more efficient.
- When intervals are very large and you need to process queries efficiently (use segment tree).

Simpler alternatives: Difference array (for range updates), Sweep Line algorithm, Segment Tree.

5. Core Concepts

Interval Representation

Typically `[start, end]` where `start ≤ end`. In C++: `pair<int,int>` or `vector<int>` of size 2.

Sorting by Start

The most common approach. Used for merging intervals and overlap detection.

Sorting by End

Used for scheduling problems (maximum number of non-overlapping intervals).

Overlap Check

Two intervals `[a, b]` and `[c, d]` overlap if `max(a, c) ≤ min(b, d)`. For sorted intervals: if `end_of_previous > start_of_current`, they overlap.

6. Step-by-Step Algorithm (Merge Intervals)

Input: intervals `[[s1,e1], [s2,e2], ..., [sn,en]]`

1. If intervals is empty, return empty.
2. Sort intervals by start time.
3. Initialize result = `[intervals[0]]`
4. For `i = 1` to `n-1`:
 - a. Let `last = result.back()`
 - b. If `intervals[i].start ≤ last.end`:
 // Overlapping: merge
 `last.end = max(last.end, intervals[i].end)`
 - c. Else:
 // Non-overlapping: add to result

```
        result.push_back(intervals[i])
5. Return result
```

7. Dry Run

Input: [[1,3], [2,6], [8,10], [15,18], [17,20]]

Step 1: Sort by start (already sorted here)

Step	Current Interval	Last in Result	Overlap?	Action	Result
Init	-	-	-	Add [1,3]	[[1,3]]
1	[2,6]	[1,3]	$2 \leq 3$	Merge $\rightarrow$ [1,6]	[[1,6]]
2	[8,10]	[1,6]	$8 \leq 6?$ No	Add [8,10]	[[1,6], [8,10]]
3	[15,18]	[8,10]	$15 \leq 10?$ No	Add [15,18]	[[1,6], [8,10], [15,18]]
4	[17,20]	[15,18]	$17 \leq 18$	Merge $\rightarrow$ [15,20]	[[1,6], [8,10], [15,20]]

Final: [[1,6], [8,10], [15,20]]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

using Interval = pair<int,int>;

// Sort by start, then by end
vector<Interval> sortByStart(vector<Interval>& intervals) {
    sort(intervals.begin(), intervals.end());
    return intervals;
}

// Sort by end, then by start
vector<Interval> sortByEnd(vector<Interval>& intervals) {
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval&
b) {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first;
    });
    return intervals;
}

// Sort by length (duration)
vector<Interval> sortByLength(vector<Interval>& intervals) {
    sort(intervals.begin(), intervals.end(), [](const Interval& a, const Interval&
b) {
        int lenA = a.second - a.first;
        int lenB = b.second - b.first;
```

```

        if (lenA != lenB) return lenA < lenB;
        return a.first < b.first;
    });
    return intervals;
}

// Merge overlapping intervals
vector<Interval> mergeIntervals(vector<Interval>& intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end()); // sort by start
    vector<Interval> result;
    result.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); ++i) {
        auto& last = result.back();
        if (intervals[i].first <= last.second) {
            // Overlapping: merge
            last.second = max(last.second, intervals[i].second);
        } else {
            result.push_back(intervals[i]);
        }
    }
    return result;
}

// Maximum number of non-overlapping intervals (Activity Selection)
int maxNonOverlapping(vector<Interval>& intervals) {
    if (intervals.empty()) return 0;

    sortByEnd(intervals); // sort by end time
    int count = 1;
    int lastEnd = intervals[0].second;

    for (int i = 1; i < intervals.size(); ++i) {
        if (intervals[i].first >= lastEnd) {
            count++;
            lastEnd = intervals[i].second;
        }
    }
    return count;
}

// Minimum number of rooms needed (Meeting Rooms II)
int minRooms(vector<Interval>& intervals) {
    if (intervals.empty()) return 0;

    // Extract start and end times
    vector<int> starts, ends;
    for (auto& interval : intervals) {
        starts.push_back(interval.first);
        ends.push_back(interval.second);
    }

```

```

    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());

    int rooms = 0, maxRooms = 0;
    int i = 0, j = 0;

    while (i < starts.size()) {
        if (starts[i] < ends[j]) {
            rooms++; // a meeting starts, need a room
            maxRooms = max(maxRooms, rooms);
            i++;
        } else {
            rooms--; // a meeting ends, free a room
            j++;
        }
    }
    return maxRooms;
}

// Insert a new interval into sorted intervals
vector<Interval> insertInterval(vector<Interval>& intervals, Interval newInterval)
{
    vector<Interval> result;
    int i = 0;
    int n = intervals.size();

    // Add all intervals ending before newInterval starts
    while (i < n && intervals[i].second < newInterval.first) {
        result.push_back(intervals[i]);
        i++;
    }

    // Merge overlapping intervals
    while (i < n && intervals[i].first <= newInterval.second) {
        newInterval.first = min(newInterval.first, intervals[i].first);
        newInterval.second = max(newInterval.second, intervals[i].second);
        i++;
    }
    result.push_back(newInterval);

    // Add remaining intervals
    while (i < n) {
        result.push_back(intervals[i]);
        i++;
    }

    return result;
}

int main() {
    vector<Interval> intervals = {{1,3}, {2,6}, {8,10}, {15,18}, {17,20}};
    auto merged = mergeIntervals(intervals);
    for (auto& [s, e] : merged) cout << "[" << s << ", " << e << "]" << " ";
    // Output: [1,6] [8,10] [15,20]
}

```

```
    return 0;
}
```

9. Python Implementation

```
from typing import List

def sort_by_start(intervals):
    """Sort by start, then by end"""
    return sorted(intervals, key=lambda x: (x[0], x[1]))

def sort_by_end(intervals):
    """Sort by end, then by start"""
    return sorted(intervals, key=lambda x: (x[1], x[0]))

def sort_by_length(intervals):
    """Sort by length (duration)"""
    return sorted(intervals, key=lambda x: (x[1] - x[0], x[0]))

def merge_intervals(intervals: List[List[int]]) -> List[List[int]]:
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0]) # sort by start
    result = [intervals[0]]

    for start, end in intervals[1:]:
        last_start, last_end = result[-1]
        if start <= last_end:
            # Overlapping: merge
            result[-1][1] = max(last_end, end)
        else:
            result.append([start, end])

    return result

def max_non_overlapping(intervals):
    """Maximum number of non-overlapping intervals (Activity Selection)"""
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1]) # sort by end
    count = 1
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= last_end:
            count += 1
            last_end = end
```

```

    return count

def min_rooms(intervals):
    """Minimum number of meeting rooms needed"""
    if not intervals:
        return 0

    starts = sorted([s for s, e in intervals])
    ends = sorted([e for s, e in intervals])

    rooms = 0
    max_rooms = 0
    i = j = 0

    while i < len(starts):
        if starts[i] < ends[j]:
            rooms += 1
            max_rooms = max(max_rooms, rooms)
            i += 1
        else:
            rooms -= 1
            j += 1

    return max_rooms

def insert_interval(intervals, new_interval):
    """Insert a new interval into sorted intervals"""
    result = []
    i = 0
    n = len(intervals)

    # Add all intervals ending before new_interval starts
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1

    # Merge overlapping intervals
    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    result.append(new_interval)

    # Add remaining
    while i < n:
        result.append(intervals[i])
        i += 1

    return result

# Example
intervals = [[1, 3], [2, 6], [8, 10], [15, 18], [17, 20]]
print(merge_intervals(intervals)) # [[1, 6], [8, 10], [15, 20]]

```

10. Code Explanation

- **Sorting by start:** Default sort on pairs works because C++ compares **first** first, then **second**.
- **Merge function:** Takes the first interval, then for each subsequent interval, either merges with the last (if overlapping) or adds as a new interval.
- **Sort by end:** Custom comparator that sorts by **second** (end time) first, then **first** (start time).
- **Activity Selection:** Greedy: always pick the interval that ends earliest. This maximizes the count.
- **Min Rooms (Meeting Rooms II):** Separates starts and ends, sorts both, and uses a sweep line. When a start is encountered before an end, we need a new room. When an end is encountered, we free a room.
- **Insert Interval:** Processes three phases: intervals before the new interval, merge overlapping, intervals after.

11. Complexity Analysis

Operation	Complexity
Sort intervals	$O(n \log n)$
Merge intervals (after sort)	$O(n)$
Activity Selection	$O(n \log n)$
Min Rooms (Meeting Rooms II)	$O(n \log n)$
Insert Interval	$O(n)$

12. Common Patterns

Merge Overlapping Intervals

- **How to identify:** "Merge overlapping intervals", "Merge meetings".
- **Approach:** Sort by start, iterate, merge if overlap.
- **Example:** LeetCode 56 "Merge Intervals"

Non-Overlapping Intervals (Activity Selection)

- **How to identify:** "Maximum number of non-overlapping intervals", "Maximum number of meetings".
- **Approach:** Sort by end, greedily pick intervals that don't overlap.
- **Example:** LeetCode 435 "Non-overlapping Intervals"

Minimum Resources (Meeting Rooms II)

- **How to identify:** "Minimum number of rooms/platforms", "Minimum resources".
- **Approach:** Sweep line: sort start and end times separately, use two pointers.
- **Example:** LeetCode 253 "Meeting Rooms II"

Insert Interval

- **How to identify:** "Insert a new interval into a sorted list".
- **Approach:** Three phases: before, merge, after.

- **Example:** LeetCode 57 "Insert Interval"

13. Common Mistakes

- **Overlap condition:** `start <= last_end` (not `<`). If one interval ends exactly where another starts, they are not overlapping for some problems (depends on definition).
- **Not updating end during merge:** `last.end = max(last.end, current.end)` — the current interval might extend beyond the previous one.
- **Sorting by start instead of end for activity selection:** Greedy by start gives wrong answer. Always sort by end for maximum count.
- **Forgetting to handle empty input.**
- **Half-open vs closed intervals:** Some problems use `[start, end)` (half-open). Adjust conditions accordingly.

14. Edge Cases

- **Empty list:** Return empty list.
- **Single interval:** Return the interval as-is.
- **All overlapping:** One merged interval.
- **No overlapping:** All intervals stay separate.
- **Adjacent intervals:** `[1, 2]` and `[2, 3]` — do they overlap? Depends on problem definition. Usually, `<=` merges them.
- **Negative values:** Intervals can have negative start/end. Works fine.
- **Unsorted input:** Always sort first.

15. Variations

Interval Tree

- A data structure for efficiently querying all intervals that overlap with a given point or interval.
- Used when you need to answer many interval queries.
- Advanced, rarely asked in placements.

Sweep Line Algorithm

- General technique for interval problems.
- Creates events: `(position, type)` where `type = +1` for start, `-1` for end.
- Sorts events and processes them.
- Used in "Meeting Rooms II", "Skyline Problem".

16. Related Algorithms/Data Structures

- **Greedy Algorithms:** Interval scheduling is a classic greedy problem.
- **Segment Tree:** For range queries and updates on intervals.
- **Difference Array:** For range updates (add value to all intervals).
- **Sweep Line:** General technique for geometric/interval problems.

17. Practice Problems

Easy

- **Merge Intervals** — LeetCode 56 (classic, must know)
- **Meeting Rooms** — LeetCode 252 (check if a person can attend all meetings)

Medium

- **Non-overlapping Intervals** — LeetCode 435 (maximum number of non-overlapping)
- **Meeting Rooms II** — LeetCode 253 (minimum number of rooms)
- **Insert Interval** — LeetCode 57 (insert and merge)

Hard

- **Minimum Interval to Include Each Query** — LeetCode 1851 (intervals + sorting + priority queue)
- **Data Stream as Disjoint Intervals** — LeetCode 352 (interval insertion with BST)
- **The Skyline Problem** — LeetCode 218 (sweep line with intervals)

18. Interview Explanation

"Sorting intervals is a preprocessing step for interval-based problems. The two most common approaches are sorting by start time (for merging) and sorting by end time (for scheduling). For merging, I sort by start, then iterate and merge if the current start $\leq$ last end. For maximum non-overlapping intervals, I sort by end and greedily pick the earliest finishing intervals. The sweep line technique, where I separate starts and ends and sort them, is useful for problems like Meeting Rooms II."

19. Revision Notes

- Sort by start $\rightarrow$ merge overlaps.
- Sort by end $\rightarrow$ activity selection (max count).
- Sweep line: separate starts and ends, sort, two pointers.
- Overlap condition: `current_start <= last_end`.
- Insert interval: three phases (before, merge, after).

20. Final Cheat Sheet

Property	Value
Sort by start	Merge intervals, detect overlaps
Sort by end	Max non-overlapping, scheduling
Sweep line	Min rooms, platform problems
Time (sort)	$O(n \log n)$
Time (after sort)	$O(n)$
Key code idea	<code>sort by start $\rightarrow$ if (start <= last_end) merge</code>
Edge cases	Empty, single, all overlap, no overlap, adjacent

9. STABLE VS UNSTABLE SORT

1. Overview

A sorting algorithm is **stable** if it preserves the relative order of elements with equal keys. An unstable sort may reorder equal elements arbitrarily.

2. Intuition

Simple explanation: Suppose you have students sorted by name, and you want to sort by grade. A stable sort will keep students with the same grade in alphabetical order (the original order). An unstable sort might not.

Analogy: Imagine a race where two runners tie for first place. The photo finish shows runner A was slightly ahead. A stable sort keeps A before B. An unstable sort might put B before A.

Step-by-step reasoning:

1. Two elements have equal keys (e.g., same value).
2. In the original array, element A appears before element B.
3. After a stable sort, A still appears before B.
4. After an unstable sort, A could appear before or after B.

Why it matters: When sorting by multiple keys, stability allows you to sort by the secondary key first, then the primary key. The second sort preserves the ordering from the first sort.

3. When to Use It

- When sorting by multiple criteria (e.g., sort by last name, then by first name).
- When the original order of equal elements matters.
- When you want to sort by one key and then by another.
- When using non-comparison sorts (Counting Sort, Radix Sort) which are naturally stable.

Trigger phrases:

- "Stable sort"
- "Preserve order"
- "Sort by multiple keys"
- "Radix sort requires stable sort"

4. When Not to Use It

- When stability is not required (e.g., sorting integers, where equal values are indistinguishable).
- When the only stable sort available is slower (e.g., using Merge Sort instead of Quick Sort for performance).
- When you can achieve the same effect by using a custom comparator that includes both keys.

Simpler alternatives: Use a custom comparator that sorts by both keys at once, avoiding the need for stability.

5. Core Concepts

Stable vs Unstable Algorithms

Stable Algorithms	Unstable Algorithms
Bubble Sort	Selection Sort
Insertion Sort	Quick Sort
Merge Sort	Heap Sort
Counting Sort	(standard)
Radix Sort	(standard)
TimSort	(standard)

Multiple Key Sorting

The classic use case for stability. To sort by primary key, then secondary key:

1. Sort by secondary key (unstable is fine here).
2. Sort by primary key (must be stable).

Since the second sort is stable, elements with the same primary key retain their secondary-key ordering from the first sort.

Practical Implications

- `std::sort()` in C++ is unstable (uses Introsort).
- `std::stable_sort()` in C++ is stable (uses Merge Sort).
- Python's `sort()` and `sorted()` are stable.
- Java's `Collections.sort()` and `Arrays.sort()` (for objects) are stable.

6. Step-by-Step Algorithm

Not an algorithm – a property of sorting algorithms.

To demonstrate stability:

1. Start with an array where some elements have equal keys.
2. Note the original order of equal elements.
3. Sort the array.
4. Check if equal elements maintain their original relative order.

7. Dry Run

Input (stable sort): Consider sorting by value only.

Original: [(1, 'A'), (2, 'B'), (2, 'C'), (1, 'D')]
 Key: first element of the pair

Stable Sort Result:

[(1, 'A'), (1, 'D'), (2, 'B'), (2, 'C')]

- All 1s: (1, 'A') before (1, 'D') — preserved.
- All 2s: (2, 'B') before (2, 'C') — preserved.

Unstable Sort Result (could be):

[(1, 'D'), (1, 'A'), (2, 'C'), (2, 'B')]

- Order of equal elements is not preserved.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Demonstrate stability difference
void demonstrateStability() {
    vector<pair<int, char>> arr = {{1, 'A'}, {2, 'B'}, {2, 'C'}, {1, 'D'}};

    // Unstable sort (std::sort)
    auto arr1 = arr;
    sort(arr1.begin(), arr1.end());
    cout << "std::sort (unstable): ";
    for (auto& [k, v] : arr1) cout << "(" << k << "," << v << ") ";
    cout << endl;
    // Output (may vary): (1,A) (1,D) (2,B) (2,C) or (1,D) (1,A) (2,C) (2,B)

    // Stable sort (std::stable_sort)
    auto arr2 = arr;
    stable_sort(arr2.begin(), arr2.end());
    cout << "std::stable_sort: ";
    for (auto& [k, v] : arr2) cout << "(" << k << "," << v << ") ";
    cout << endl;
    // Output: (1,A) (1,D) (2,B) (2,C) -- always stable
}

// Sorting by multiple keys using stability
void multiKeySorting() {
    // Sort by grade first, then by name (both alphabetically)
    vector<pair<string, int>> students = {
```

```

        {"Alice", 85}, {"Bob", 75}, {"Charlie", 85}, {"David", 90}
    };

    // Step 1: Sort by name (secondary key)
    sort(students.begin(), students.end());

    // Step 2: Sort by grade (primary key) using stable_sort
    stable_sort(students.begin(), students.end(), [](auto& a, auto& b) {
        return a.second < b.second;
    });

    cout << "Sorted by grade, then by name: ";
    for (auto& [name, grade] : students) {
        cout << name << "(" << grade << ") ";
    }
    cout << endl;
    // Output: Bob(75) Alice(85) Charlie(85) David(90)
    // Note: Alice before Charlie because step 1 sorted by name
}

// Check if a sort is stable
template<typename T>
bool isStableSort(const vector<T>& original, const vector<T>& sorted) {
    // Build a map from element to list of original positions
    unordered_map<T, vector<int>> positions;
    for (int i = 0; i < original.size(); ++i) {
        positions[original[i]].push_back(i);
    }

    // Check that the relative order of equal elements is preserved
    unordered_map<T, int> nextIndex;
    for (int i = 0; i < sorted.size(); ++i) {
        int expectedPos = positions[sorted[i]][nextIndex[sorted[i]]++];
        // We can't easily check without knowing equal elements
    }
    return true; // simplified
}

int main() {
    demonstrateStability();
    multiKeySorting();
    return 0;
}

```

9. Python Implementation

```

# Python's sort is stable
arr = [(1, 'A'), (2, 'B'), (2, 'C'), (1, 'D')]
arr.sort(key=lambda x: x[0])
print(arr) # [(1, 'A'), (1, 'D'), (2, 'B'), (2, 'C')] -- always stable

```

```
# Multi-key sorting using stability
students = [("Alice", 85), ("Bob", 75), ("Charlie", 85), ("David", 90)]

# Step 1: Sort by name (secondary key)
students.sort(key=lambda x: x[0])

# Step 2: Sort by grade (primary key) - stable sort preserves name order
students.sort(key=lambda x: x[1])

print(students) # [('Bob', 75), ('Alice', 85), ('Charlie', 85), ('David', 90)]
```

10. Code Explanation

- **std::sort**: Introsort — hybrid of Quick Sort, Heap Sort, and Insertion Sort. Unstable.
- **std::stable_sort**: Merge Sort-based. Stable. Uses O(n) extra space or O(n log n) time.
- **Python's sort**: TimSort — hybrid of Merge Sort and Insertion Sort. Stable.
- **Multi-key sorting**: Sort by secondary key first (any sort), then sort by primary key using a stable sort.

11. Complexity Analysis

Algorithm	Time	Space	Stable
std::sort	O(n log n)	O(log n)	No
std::stable_sort	O(n log n)	O(n)	Yes
Python sort	O(n log n)	O(n)	Yes

12. Common Patterns

Radix Sort

- **Requires stable sort** as a subroutine.
- Counting Sort (stable) is used for each digit.
- If an unstable sort is used, Radix Sort fails.

Sorting by Multiple Keys

- **Approach**: Sort by secondary key, then stable-sort by primary key.
- **Alternative**: Use a custom comparator that compares both keys.
- **Example**: "Sort by last name, then first name"

Maintaining Original Order of Ties

- **Approach**: Use a stable sort with a custom comparator.
- **Example**: "Sort by score descending, but keep alphabetical order for ties"

13. Common Mistakes

- **Assuming C++ std::sort is stable**: It's not. Use std::stable_sort if stability is needed.
- **Assuming Python's sort is unstable**: It's not. Python's sort is stable.

- **Using unstable sort for Radix Sort:** Radix Sort requires a stable digit sort.
- **Not understanding the practical impact of instability:** For integers, instability doesn't matter. For objects with multiple fields, it can.

14. Edge Cases

- **All equal elements:** Stable sort preserves original order. Unstable sort may reorder.
- **No equal elements:** Stability is irrelevant.
- **Single element:** Trivially stable.
- **Empty array:** Trivially stable.

15. Variations

In-Place Stable Sort

- Can be done with $O(1)$ space using "block swap" merge.
- Rarely used in practice due to complexity.
- Not important for placements.

TimSort

- A stable, adaptive, hybrid sort.
- $O(n)$ for nearly sorted data, $O(n \log n)$ worst case.
- Used in Python, Java, and other languages.

16. Related Algorithms/Data Structures

- **Sorting Algorithms:** All sorting algorithms have a stability property.
- **Radix Sort:** Requires a stable sort (Counting Sort) for each digit.

17. Practice Problems

Easy

- **Sort by Color** — LeetCode 75 (stable sort not required, but good to test)
- **Sort the People** — LeetCode 2418 (sort by height, keep name order)

Medium

- **Sort Characters By Frequency** — LeetCode 451 (stable sort for equal frequency)
- **Relative Sort Array** — LeetCode 1122 (stable sort with custom ordering)

Hard

- **Maximum Number of Events That Can Be Attended II** — LeetCode 1751 (stable sort for DP)

18. Interview Explanation

"A stable sort preserves the relative order of elements with equal keys. `std::sort` in C++ is unstable (uses Introsort), while `std::stable_sort` is stable (uses Merge Sort). Python's sort is stable (uses

TimSort). Stability is important when sorting by multiple keys — you sort by the secondary key first, then use a stable sort for the primary key. Radix Sort also requires a stable digit sort."

19. Revision Notes

- Stable: equal elements keep original order.
- C++ `std::sort` → unstable. `std::stable_sort` → stable.
- Python `sort()` → stable.
- Multi-key: sort secondary first, then stable-sort primary.
- Radix Sort needs stable digit sort.

20. Final Cheat Sheet

Property	Value
Stable sorts	Bubble, Insertion, Merge, Counting, Radix, TimSort
Unstable sorts	Selection, Quick, Heap
C++ <code>sort</code>	Unstable (Introsort)
C++ <code>stable_sort</code>	Stable (Merge Sort)
Python <code>sort</code>	Stable (TimSort)
Key code idea	<code>std::stable_sort</code> for stability in C++
Edge cases	All equal, no equal, single, empty

10. RADIX SORT

1. Overview

Radix Sort is a non-comparison sorting algorithm that sorts numbers by processing individual digits. It sorts numbers digit by digit, from the least significant digit (LSD) to the most significant digit (MSD), using a stable sort (usually Counting Sort) for each digit position.

2. Intuition

Simple explanation: Instead of comparing numbers, sort them by their last digit, then by the second-to-last digit, and so on. After processing all digits, the numbers are fully sorted.

Analogy: Imagine sorting mail by zip code. You first sort by the last digit, then by the second-to-last, and so on. After sorting by all digits, the mail is in zip code order.

Step-by-step reasoning:

1. Find the maximum number to determine the number of digits.
2. For each digit position (from least significant to most significant):
a. Use Counting Sort to sort by the current digit.
b. Counting Sort must be stable (preserves order from previous digit sorts).
3. After processing all digits, the array is sorted.

Why it works: After sorting by the least significant digit, numbers are sorted by that digit. After sorting by the next digit, numbers are sorted by the last two digits (because the sort is stable, the order from the previous digit is preserved when the current digit is the same). After processing all digits, the full number is sorted.

3. When to Use It

- When sorting integers with a fixed number of digits (e.g., 32-bit integers, strings, dates).
- When the range of values is large but the number of digits is small.
- When you need $O(n \cdot k)$ time where k is the number of digits ($k \ll n$).
- When you need a stable, non-comparison sort.
- When sorting strings of equal length.

Trigger phrases:

- "Linear time sort for integers"
- "Sort by digits"
- "Non-comparison sort"
- "Sort strings"

4. When Not to Use It

- When the number of digits is large (e.g., sorting 64-bit numbers with 19 digits — Counting Sort with range 10 is fine, but the number of passes is large).
- When sorting floating-point numbers directly (requires bit manipulation).
- When the range of values is small (Counting Sort alone is simpler).
- When the input is small (comparison sorts are simpler and fast enough).
- When the base is large (e.g., base 2^{16} for 16-bit digits — the count array is large).

Simpler alternatives: Counting Sort (for small range), `sort()` (for general use).

5. Core Concepts

Base (Radix)

The number base used for digit extraction. Usually base 10 (decimal digits), but base 2^8 (byte), base 2^{16} (half-word), or base 256 (ASCII characters) are common.

LSD vs MSD

- **LSD (Least Significant Digit):** Starts from the rightmost digit. More common and simpler.
- **MSD (Most Significant Digit):** Starts from the leftmost digit. Used for strings and lexicographic sorting.

Stable Subroutine

Radix Sort requires a stable sort for each digit. Counting Sort is the most common choice because it's stable and $O(n)$ for a small range (the base).

Digit Extraction

Extracting the k-th digit: $(\text{num} / \text{base}^k) \% \text{base}$.

6. Step-by-Step Algorithm (LSD Radix Sort)

Input: `arr[0..n-1]` of non-negative integers

- `maxVal` = max element in `arr`
- `exp` = 1 (current digit position: 1s, 10s, 100s, ...)
- While `maxVal / exp > 0`:
 - Use Counting Sort to sort `arr` by $(\text{arr}[i] / \text{exp}) \% 10$
 - `exp` = `exp * 10`

7. Dry Run

Input: `[170, 45, 75, 90, 2, 802, 24, 66]` `maxVal` = 802, digits = 3

Pass 1 (`exp` = 1, sorting by 1s digit):

Original	Digit (1s)	After Counting Sort
170	0	170
45	5	90
75	5	2
90	0	802
2	2	24
802	2	45
24	4	75
66	6	66

After pass 1: `[170, 90, 2, 802, 24, 45, 75, 66]`

Pass 2 (`exp` = 10, sorting by 10s digit):

Before Pass 2	Digit (10s)	After Counting Sort
170	7	2
90	9	802
2	0	24
802	0	45
24	2	66
45	4	170

Before Pass 2	Digit (10s)	After Counting Sort
75	7	75
66	6	90

After pass 2: [2, 802, 24, 45, 66, 170, 75, 90]

Pass 3 (exp = 100, sorting by 100s digit):

Before Pass 3	Digit (100s)	After Counting Sort
2	0	2
802	8	24
24	0	45
45	0	66
66	0	75
170	1	90
75	0	170
90	0	802

Final: [2, 24, 45, 66, 75, 90, 170, 802]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Counting Sort for a specific digit (exp = 1, 10, 100, ...)
void countSortByDigit(vector<int>& arr, int exp) {
    int n = arr.size();
    vector<int> output(n);
    int count[10] = {0};

    // Count occurrences of each digit
    for (int i = 0; i < n; ++i) {
        count[(arr[i] / exp) % 10]++;
    }

    // Cumulative count
    for (int i = 1; i < 10; ++i) {
        count[i] += count[i - 1];
    }

    // Build output array (stable sort, right to left)
    for (int i = n - 1; i >= 0; --i) {
        int digit = (arr[i] / exp) % 10;
```

```
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    // Copy back
    arr = output;
}

void radixSort(vector<int>& arr) {
    if (arr.empty()) return;

    // Find maximum to know number of digits
    int maxVal = *max_element(arr.begin(), arr.end());

    // Handle negative numbers: shift all values to non-negative
    int minVal = *min_element(arr.begin(), arr.end());
    if (minVal < 0) {
        for (int& x : arr) x -= minVal;
        maxVal -= minVal;
    }

    // Sort by each digit (LSD to MSD)
    for (int exp = 1; maxVal / exp > 0; exp *= 10) {
        countSortByDigit(arr, exp);
    }

    // Shift back if negative values were present
    if (minVal < 0) {
        for (int& x : arr) x += minVal;
    }
}

// Optimized version with variable base (e.g., base 256 for bytes)
void radixSortBase256(vector<int>& arr) {
    if (arr.empty()) return;

    // Sort by bytes (4 passes for 32-bit integers)
    const int BITS = 8;
    const int BASE = 1 << BITS; // 256
    const int MASK = BASE - 1;

    vector<int> output(arr.size());

    for (int shift = 0; shift < 32; shift += BITS) {
        vector<int> count(BASE, 0);

        // Count
        for (int x : arr) {
            count[(x >> shift) & MASK]++;
        }

        // Cumulative
        for (int i = 1; i < BASE; ++i) {
            count[i] += count[i - 1];
        }
    }
}
```

```

    }

    // Place (stable, right to left)
    for (int i = arr.size() - 1; i >= 0; --i) {
        int digit = (arr[i] >> shift) & MASK;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    arr = output;
}

int main() {
    vector<int> arr = {170, 45, 75, 90, 2, 802, 24, 66};
    radixSort(arr);
    for (int x : arr) cout << x << " ";
    // Output: 2 24 45 66 75 90 170 802
    return 0;
}

```

9. Python Implementation

```

def counting_sort_by_digit(arr, exp):
    """Counting Sort for a specific digit (exp = 1, 10, 100, ...)"""
    n = len(arr)
    output = [0] * n
    count = [0] * 10

    # Count occurrences of each digit
    for num in arr:
        digit = (num // exp) % 10
        count[digit] += 1

    # Cumulative count
    for i in range(1, 10):
        count[i] += count[i - 1]

    # Build output array (stable, right to left)
    for i in range(n - 1, -1, -1):
        digit = (arr[i] // exp) % 10
        output[count[digit] - 1] = arr[i]
        count[digit] -= 1

    return output

def radix_sort(arr):
    if not arr:
        return arr

    # Handle negative numbers

```

```
min_val = min(arr)
if min_val < 0:
    arr = [x - min_val for x in arr]

max_val = max(arr)
exp = 1

while max_val // exp > 0:
    arr = counting_sort_by_digit(arr, exp)
    exp *= 10

# Shift back
if min_val < 0:
    arr = [x + min_val for x in arr]

return arr

arr = [170, 45, 75, 90, 2, 802, 24, 66]
print(radix_sort(arr)) # [2, 24, 45, 66, 75, 90, 170, 802]
```

10. Code Explanation

- **countSortByDigit**: A Counting Sort variant that sorts by a specific digit position.
- **exp**: The digit position (1 = units, 10 = tens, 100 = hundreds, ...).
- **Digit extraction**: $(arr[i] / exp) \% 10$ gives the digit at position **exp**.
- **Stable sort**: Right-to-left iteration ensures stability.
- **Negative handling**: Shift all values to non-negative by subtracting the minimum, sort, then shift back.
- **Base 256 version**: Sorts 32-bit integers in 4 passes (8 bits per pass). More efficient for large arrays.

11. Complexity Analysis

Operation	Complexity
Time	$O(d \cdot (n + b))$
Space	$O(n + b)$
Stable	Yes

For base 10: $O(d \cdot (n + 10)) = O(d \cdot n)$
For base 256 (32-bit ints): $O(4 \cdot (n + 256)) = O(n)$
When $d = O(\log n)$: Radix Sort is $O(n \log n)$ — same as comparison sorts.

12. Common Patterns

Sorting Strings

- **How to identify**: Sort strings of equal length lexicographically.
- **Approach**: MSD Radix Sort (start from the most significant character).
- **Example**: "Sort an array of strings"

Sorting Large Integers

- **How to identify:** Sort integers with a large range but limited digits.
- **Approach:** LSD Radix Sort with base 256 (byte by byte).
- **Example:** "Sort 10^6 integers in range $[0, 10^9]$ "

Sort by Multiple Fields

- **How to identify:** Sort by one field, then another.
- **Approach:** Use Radix Sort on the concatenated fields (requires stable sorting).
- **Example:** "Sort by date (year, month, day)"

13. Common Mistakes

- **Not stable digit sort:** Radix Sort requires a stable sort for each digit. Using an unstable sort breaks the algorithm.
- **Wrong digit extraction:** For negative numbers, division and modulo behave differently in C++ (truncation toward zero vs floor division).
- **Forgetting to handle negative numbers:** Standard Radix Sort works for non-negative integers only.
- **Incorrect base:** Using base 10 for large numbers means many passes. Use base 256 for 32-bit integers (4 passes).
- **Off-by-one in digit extraction:** $(num / 10^k) \% 10$ gives the k-th digit from the right (0-indexed).

14. Edge Cases

- **Empty array:** Return immediately.
- **Single element:** Already sorted.
- **All zeros:** All digits are 0, each pass is a no-op.
- **Negative numbers:** Must be handled by shifting or using absolute values.
- **Large range but few digits:** Radix Sort works well (e.g., 10^6 numbers with 3 digits each).
- **Numbers with different lengths:** Pad with leading zeros (effectively).

15. Variations

MSD Radix Sort

- Starts from the most significant digit.
- Used for lexicographic sorting of strings.
- Can be recursive (process each bucket independently).
- More complex than LSD.

In-Place Radix Sort

- Uses $O(1)$ extra space.
- More complex; rarely used in practice.
- Not important for placements.

American Flag Sort

- An in-place variant of MSD Radix Sort.

- Efficient for sorting strings.

16. Related Algorithms/Data Structures

- **Counting Sort:** Used as the subroutine for each digit.
- **Bucket Sort:** Another non-comparison sort. Radix Sort is sometimes considered a variant of Bucket Sort.
- **TimSort / Introsort:** Comparison-based sorts that are faster on average for general-purpose sorting.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Radix Sort for practice)
- **Maximum Gap** — LeetCode 164 (Radix Sort / Bucket Sort)

Medium

- **Sort Colors** — LeetCode 75 (trivial, but Radix Sort works with base 3)
- **Sorting the Sentence** — LeetCode 1859 (Radix Sort by position)

Hard

- **Radix Sort with Negative Numbers** — GFG (implement negative handling)
- **Sort Array by Moving Items to Empty Space** — Codeforces

18. Interview Explanation

"Radix Sort is a non-comparison integer sort that processes digits from least significant to most significant. It uses a stable sort, typically Counting Sort, for each digit position. After processing all digits, the array is fully sorted. It runs in $O(d \cdot n)$ time where d is the number of digits, and uses $O(n)$ space. It's efficient when d is small, but handling negative numbers requires shifting. It's a good choice for sorting large arrays of integers with limited digits."

19. Revision Notes

- LSD Radix Sort: sort by units, tens, hundreds, ...
- Requires stable digit sort (Counting Sort).
- Complexity: $O(d \cdot n)$, space: $O(n)$.
- Handle negatives by shifting.
- Base 256 for 32-bit integers $\rightarrow$ 4 passes.
- Not comparison-based — can beat $O(n \log n)$.

20. Final Cheat Sheet

Property	Value
When to use	Large integers, limited digits, strings
Type	Non-comparison, stable

Property	Value
Time	$O(d \cdot n)$
Space	$O(n)$
Stable	Yes
Key code idea	<code>countSortByDigit(arr, exp)</code> for $\text{exp} = 1, 10, 100, \dots$
Edge cases	Empty, single, all zeros, negative, different lengths

11. BUCKET SORT

1. Overview

Bucket Sort distributes elements into a number of "buckets" based on their value range, sorts each bucket individually (using another sorting algorithm), and then concatenates the buckets in order.

2. Intuition

Simple explanation: Divide the range of values into equal-sized buckets. Put each element into the appropriate bucket. Sort each bucket individually. Concatenate the buckets.

Analogy: Imagine sorting exam scores (0–100). You create 10 buckets: 0–9, 10–19, ..., 90–100. Put each score in the correct bucket. Sort each bucket (they're small, so it's fast). Then concatenate the buckets.

Step-by-step reasoning:

1. Create k empty buckets.
2. Scatter: Distribute elements into buckets based on their value.
3. Sort each bucket individually (usually Insertion Sort or built-in sort).
4. Concatenate: Gather elements from buckets in order.

Why it works: If the elements are uniformly distributed, each bucket has roughly n/k elements. Sorting each bucket takes $O((n/k) \log(n/k))$ time. With $k \approx n$, each bucket has $O(1)$ elements, giving $O(n)$ total time.

3. When to Use It

- When input is uniformly distributed over a known range.
- When sorting floating-point numbers in the range $[0, 1)$.
- When you need a linear-time sort for non-integer data.
- When the data can be mapped to buckets efficiently.
- As a subroutine for more complex algorithms.

Trigger phrases:

- "Uniformly distributed data"
- "Floating point sort"
- "Linear time sort"

- "Sort by range"

4. When Not to Use It

- When the input is not uniformly distributed (all elements in one bucket $\rightarrow O(n^2)$).
- When the range is very large and the number of buckets is small.
- When memory is constrained (buckets add overhead).
- When the data is already sorted or nearly sorted (Insertion Sort alone is better).
- When the distribution is unknown or unpredictable.

Simpler alternatives: Counting Sort (for integers with small range), Radix Sort (for integers), `sort()` (general purpose).

5. Core Concepts

Number of Buckets

Typically $k = n$ (number of elements) or $k = \sqrt{n}$. More buckets mean smaller buckets but more overhead.

Bucket Range

Each bucket covers a range of $(\text{max} - \text{min}) / k$. The i -th bucket covers $[\text{min} + i \cdot \text{range}, \text{min} + (i+1) \cdot \text{range})$.

Scatter Phase

Distributing elements into buckets. This is $O(n)$ if the bucket index can be computed in $O(1)$.

Sort Phase

Sorting each bucket. The choice of sorting algorithm depends on the expected bucket size.

Concatenate Phase

Gathering elements from buckets in order. This is $O(n)$.

6. Step-by-Step Algorithm

Input: `arr[0..n-1]` of floating-point numbers in range `[0, 1)`

1. Create n empty buckets.
2. For $i = 0$ to $n-1$:
 `bucketIdx = floor(n * arr[i])`
 Insert `arr[i]` into `bucket[bucketIdx]`
3. For $i = 0$ to $n-1$:
 Sort `bucket[i]` (e.g., using Insertion Sort)
4. Concatenate all buckets in order into the output array.

7. Dry Run

Input: [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51] (n = 7)

Scatter (n = 7 buckets):

Element	bucketIdx = floor(7 * element)	Bucket
0.42	2	2
0.32	2	2
0.23	1	1
0.52	3	3
0.25	1	1
0.47	3	3
0.51	3	3

Buckets:

Bucket	Elements
0	[]
1	[0.23, 0.25]
2	[0.42, 0.32]
3	[0.52, 0.47, 0.51]
4	[]
5	[]
6	[]

Sort each bucket:

Bucket	Sorted
1	[0.23, 0.25]
2	[0.32, 0.42]
3	[0.47, 0.51, 0.52]

Concatenate: [0.23, 0.25, 0.32, 0.42, 0.47, 0.51, 0.52]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Bucket sort for floating-point numbers in range [0, 1)
void bucketSort(vector<double>& arr) {
    int n = arr.size();
    if (n <= 1) return;

    // Create n empty buckets
    vector<vector<double>> buckets(n);

    // Scatter: put elements into buckets
    for (double x : arr) {
        int bucketIdx = n * x; // floor(n * x) since x < 1
        buckets[bucketIdx].push_back(x);
    }

    // Sort each bucket
    for (auto& bucket : buckets) {
        sort(bucket.begin(), bucket.end());
    }

    // Concatenate
    int idx = 0;
    for (auto& bucket : buckets) {
        for (double x : bucket) {
            arr[idx++] = x;
        }
    }
}

// Bucket sort for integers with a given range
void bucketSortInt(vector<int>& arr, int minVal, int maxVal) {
    int n = arr.size();
    if (n <= 1) return;

    int range = maxVal - minVal + 1;
    int bucketCount = sqrt(n); // or use n, or range
    if (bucketCount == 0) bucketCount = 1;

    vector<vector<int>> buckets(bucketCount);

    // Scatter
    for (int x : arr) {
        int bucketIdx = (long long)(x - minVal) * bucketCount / range;
        if (bucketIdx >= bucketCount) bucketIdx = bucketCount - 1;
        buckets[bucketIdx].push_back(x);
    }

    // Sort each bucket
    for (auto& bucket : buckets) {
        sort(bucket.begin(), bucket.end());
    }

    // Concatenate
    int idx = 0;
```

```

    for (auto& bucket : buckets) {
        for (int x : bucket) {
            arr[idx++] = x;
        }
    }
}

int main() {
    vector<double> arr = {0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51};
    bucketSort(arr);
    for (double x : arr) cout << x << " ";
    // Output: 0.23 0.25 0.32 0.42 0.47 0.51 0.52
    return 0;
}

```

9. Python Implementation

```

import math

def bucket_sort(arr):
    """Bucket sort for floating-point numbers in range [0, 1)"""
    n = len(arr)
    if n <= 1:
        return arr

    # Create n empty buckets
    buckets = [[] for _ in range(n)]

    # Scatter: put elements into buckets
    for x in arr:
        bucket_idx = int(n * x) # floor(n * x) since x < 1
        buckets[bucket_idx].append(x)

    # Sort each bucket
    for bucket in buckets:
        bucket.sort()

    # Concatenate
    result = []
    for bucket in buckets:
        result.extend(bucket)

    return result

def bucket_sort_int(arr, min_val, max_val):
    """Bucket sort for integers with a given range"""
    n = len(arr)
    if n <= 1:
        return arr

    range_val = max_val - min_val + 1

```

```

bucket_count = max(1, int(math.sqrt(n)))

buckets = [[] for _ in range(bucket_count)]

# Scatter
for x in arr:
    bucket_idx = (x - min_val) * bucket_count // range_val
    if bucket_idx >= bucket_count:
        bucket_idx = bucket_count - 1
    buckets[bucket_idx].append(x)

# Sort each bucket
for bucket in buckets:
    bucket.sort()

# Concatenate
result = []
for bucket in buckets:
    result.extend(bucket)

return result

arr = [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51]
print(bucket_sort(arr)) # [0.23, 0.25, 0.32, 0.42, 0.47, 0.51, 0.52]

```

10. Code Explanation

- **Bucket creation:** `vector<vector<double>> buckets(n)` creates `n` empty buckets.
- **Scatter:** `bucketIdx = n * x` maps the value to a bucket index. Works for values in `[0, 1)`.
- **Sort each bucket:** Using `sort()` (or Insertion Sort for small buckets).
- **Concatenate:** Iterate through buckets in order, collecting all elements.
- **Integer version:** Computes the bucket index based on the value's position in the range.

11. Complexity Analysis

Operation	Complexity
Time (Average)	$O(n + k)$
Time (Worst)	$O(n^2)$
Space	$O(n + k)$
Stable	Yes

12. Common Patterns

Sort Floating-Point Numbers

- **How to identify:** Sort numbers in `[0, 1)` or any known range.
- **Approach:** Bucket Sort with `n` buckets.
- **Example:** "Sort an array of doubles uniformly distributed in `[0, 1)`"

Sort by Frequency (Bucket Sort Variant)

- **How to identify:** Sort by frequency of elements.
- **Approach:** Count frequencies, use bucket sort where buckets are frequencies.
- **Example:** LeetCode 451 "Sort Characters By Frequency"

Maximum Gap

- **How to identify:** Find the maximum difference between consecutive elements after sorting.
- **Approach:** Use buckets to find the gap without fully sorting.
- **Example:** LeetCode 164 "Maximum Gap"

13. Common Mistakes

- **Wrong bucket index calculation:** For values in $[0, 1)$, $\text{bucketIdx} = n * x$ (not $n * x - 1$).
- **Not handling the edge case $x = 1.0$:** If $x = 1.0$, $n * x = n$, which is out of bounds. Handle separately or use $\min(n-1, n*x)$.
- **Assuming uniform distribution:** Worst-case is $O(n^2)$ if all elements go to the same bucket.
- **Too many buckets:** More buckets = more overhead. The sweet spot is typically n or $\sqrt{n}$.
- **Using an expensive sort for buckets:** For small buckets, Insertion Sort is better than Merge Sort.

14. Edge Cases

- **Empty array:** Return immediately.
- **Single element:** Already sorted.
- **All elements equal:** All go to the same bucket $\rightarrow O(n^2)$ with Insertion Sort.
- **All elements at boundaries:** Values exactly at 0.0 or 1.0 need special handling.
- **Non-uniform distribution:** Performance degrades toward $O(n^2)$.

15. Variations

Generic Bucket Sort

- Allows custom hashing function to map elements to buckets.
- Used for sorting based on a key (e.g., sort by last name initial).

Histogram Sort

- Uses bucket counts to determine positions (similar to Counting Sort).
- More efficient for integer data.

16. Related Algorithms/Data Structures

- **Counting Sort:** Special case of Bucket Sort where each bucket has size 1.
- **Radix Sort:** Uses buckets for each digit position.
- **Quick Sort:** Can be seen as a recursive two-bucket sort (partitioning).

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (implement Bucket Sort for practice)
- **Sort Characters By Frequency** — LeetCode 451 (bucket by frequency)

Medium

- **Maximum Gap** — LeetCode 164 (bucket-based linear time solution)
- **Top K Frequent Elements** — LeetCode 347 (bucket sort by frequency)

Hard

- **Sorting with Custom Comparator** — GFG (combine with bucket sort)
- **The Skyline Problem** — LeetCode 218 (sweep line + bucket concepts)

18. Interview Explanation

"Bucket Sort distributes elements into buckets based on their value range, sorts each bucket individually, and concatenates them. It's efficient when the input is uniformly distributed, giving $O(n)$ average time. The worst case is $O(n^2)$ when all elements go to the same bucket. It's commonly used for sorting floating-point numbers in $[0, 1)$ and as a subroutine in problems like Maximum Gap."

19. Revision Notes

- Scatter → sort each bucket → concatenate.
- $O(n)$ average if uniformly distributed, $O(n^2)$ worst case.
- Good for floating-point numbers in $[0, 1)$.
- Use n buckets, `bucketIdx = floor(n * x)`.
- Not suitable for non-uniform distributions.

20. Final Cheat Sheet

Property	Value
When to use	Uniformly distributed data, floating-point, known range
Type	Non-comparison, distribution-based
Time	$O(n)$ avg, $O(n^2)$ worst
Space	$O(n)$
Stable	Yes (with stable per-bucket sort)
Key code idea	<code>buckets[n * x].push_back(x); sort(bucket); concat</code>
Edge cases	Empty, single, all equal, boundaries, non-uniform

12. TOPOLOGICAL SORTING

1. Overview

Topological Sort is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u appears before v in the ordering.

2. Intuition

Simple explanation: If you have a set of tasks where some tasks depend on others, topological sort gives you a valid order to complete all tasks. You must complete prerequisites before the tasks that depend on them.

Analogy: University course prerequisites. To take "Advanced Algorithms", you need "Data Structures" first. To take "Data Structures", you need "Programming Fundamentals" first. Topological sort gives the order: Programming Fundamentals $\rightarrow$ Data Structures $\rightarrow$ Advanced Algorithms.

Step-by-step reasoning (Kahn's Algorithm):

1. Find all vertices with no incoming edges (no prerequisites).
2. Add them to a queue.
3. While the queue is not empty: a. Remove a vertex from the queue and add it to the result. b. For each outgoing edge from this vertex, reduce the in-degree of the target vertex. c. If the target vertex's in-degree becomes 0, add it to the queue.
4. If the result contains all vertices, the graph is a DAG and we have a topological order. Otherwise, the graph has a cycle.

Why it works: Kahn's algorithm greedily removes vertices that have no remaining dependencies. Since we only process vertices whose in-degree is 0, we never place a vertex before its prerequisites.

3. When to Use It

- When you need to find a valid order of tasks with dependencies.
- When detecting cycles in a directed graph.
- When scheduling tasks with prerequisites.
- When resolving dependency graphs (package managers, build systems).
- When finding the longest path in a DAG (using topological sort + DP).

Trigger phrases:

- "Course schedule"
- "Task scheduling with dependencies"
- "Build order"
- "Prerequisites"
- "Dependency resolution"
- "Detect cycle in directed graph"
- "Longest path in DAG"

4. When Not to Use It

- When the graph is not a DAG (has cycles) — topological sort is not defined for cyclic graphs.
- When the graph is undirected (use other algorithms like BFS/DFS for connectivity).
- When you need any valid ordering (not specifically a topological one).
- When the graph is very small (simple DFS recursion is fine).

Simpler alternatives: DFS with a stack (for small graphs), Kahn's algorithm (for iterative approach).

5. Core Concepts

Directed Acyclic Graph (DAG)

A directed graph with no cycles. Topological sort only exists for DAGs.

In-Degree

The number of incoming edges to a vertex. Vertices with in-degree 0 have no prerequisites and can be processed first.

Kahn's Algorithm (BFS-based)

Uses a queue of vertices with in-degree 0. Removes vertices and decrements the in-degree of their neighbors.

DFS-based Topological Sort

Uses DFS with a stack. When DFS finishes exploring a vertex, push it to the stack. The stack (in reverse) gives the topological order.

Cycle Detection

If the result of Kahn's algorithm doesn't contain all vertices, the graph has a cycle.

6. Step-by-Step Algorithm

Kahn's Algorithm (BFS)

```
Input: Graph with V vertices and adjacency list adj

1. Compute in-degree for each vertex.
2. Initialize queue with all vertices having in-degree = 0.
3. result = []
4. While queue is not empty:
    u = queue.pop()
    result.push(u)
    For each v in adj[u]:
        in-degree[v]--
        If in-degree[v] == 0:
            queue.push(v)
5. If result.size() != V: graph has a cycle
6. Return result
```

DFS-based Algorithm

```
Input: Graph with V vertices and adjacency list adj
```

```

1. visited = array of size V, initialized to false
2. stack = [] (to store the topological order)
3. For each vertex u from 0 to V-1:
    If not visited[u]:
        dfs(u, visited, stack, adj)

dfs(u, visited, stack, adj):
    visited[u] = true
    For each v in adj[u]:
        If not visited[v]:
            dfs(v, visited, stack, adj)
    stack.push(u) // post-order

4. Reverse the stack to get the topological order.

```

7. Dry Run

Graph: 6 vertices (0-5), edges: 5→2, 5→0, 4→0, 4→1, 2→3, 3→1

```

5 → 2 → 3
↓       ↓
0       1
↑
4

```

Kahn's Algorithm:

Initial in-degrees:

Vertex	0	1	2	3	4	5
In-degree	2	2	1	1	0	0

Queue (in-degree = 0): [4, 5]

Step	Remove	Adjacent	Decrement	Queue (after)	Result
1	4	0, 1	in[0]=1, in[1]=1	[5]	[4]
2	5	2, 0	in[2]=0, in[0]=0	[2, 0]	[4, 5]
3	2	3	in[3]=0	[0, 3]	[4, 5, 2]
4	0	-	-	[3]	[4, 5, 2, 0]
5	3	1	in[1]=0	[1]	[4, 5, 2, 0, 3]
6	1	-	-	[]	[4, 5, 2, 0, 3, 1]

Topological Order: [4, 5, 2, 0, 3, 1]

Verification: Every edge $u \rightarrow v$ has u before v in the ordering.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Kahn's Algorithm (BFS-based)
vector<int> topologicalSortKahn(int V, vector<vector<int>>& adj) {
    vector<int> inDegree(V, 0);

    // Compute in-degrees
    for (int u = 0; u < V; ++u) {
        for (int v : adj[u]) {
            inDegree[v]++;
        }
    }

    // Queue for vertices with in-degree 0
    queue<int> q;
    for (int i = 0; i < V; ++i) {
        if (inDegree[i] == 0) q.push(i);
    }

    vector<int> result;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        result.push_back(u);

        for (int v : adj[u]) {
            inDegree[v]--;
            if (inDegree[v] == 0) q.push(v);
        }
    }

    // If result doesn't contain all vertices, graph has a cycle
    if (result.size() != V) {
        return {}; // cycle detected
    }

    return result;
}

// DFS-based Topological Sort
void dfs(int u, vector<bool>& visited, stack<int>& st, vector<vector<int>>& adj) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, visited, st, adj);
        }
    }
    st.push(u);
}

```

```

vector<int> topologicalSortDFS(int V, vector<vector<int>>& adj) {
    vector<bool> visited(V, false);
    stack<int> st;

    for (int i = 0; i < V; ++i) {
        if (!visited[i]) {
            dfs(i, visited, st, adj);
        }
    }

    vector<int> result;
    while (!st.empty()) {
        result.push_back(st.top());
        st.pop();
    }
    return result;
}

// Cycle detection using Kahn's algorithm
bool hasCycle(int V, vector<vector<int>>& adj) {
    auto result = topologicalSortKahn(V, adj);
    return result.empty();
}

// Find longest path in DAG
vector<int> longestPath(int V, vector<vector<pair<int,int>>>& adj, int src) {
    // adj[u] = {(v, weight), ...}
    vector<int> topo = topologicalSortKahn(V, vector<vector<int>>(V));
    // Convert to adjacency without weights if needed

    // DP: distance to each vertex
    vector<int> dist(V, INT_MIN);
    dist[src] = 0;

    for (int u : topo) {
        if (dist[u] != INT_MIN) {
            for (auto& [v, w] : adj[u]) {
                if (dist[u] + w > dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }

    return dist;
}

int main() {
    int V = 6;
    vector<vector<int>> adj(V);
    adj[5] = {2, 0};
    adj[4] = {0, 1};
    adj[2] = {3};
}

```

```

adj[3] = {1};

vector<int> order = topologicalSortKahn(V, adj);
cout << "Topological Order: ";
for (int v : order) cout << v << " ";
// Output: 4 5 2 0 3 1 (or any valid topological order)
cout << endl;

// Check if cycle exists
cout << "Has cycle: " << (hasCycle(V, adj) ? "Yes" : "No") << endl;
// Output: Has cycle: No

return 0;
}

```

9. Python Implementation

```

from collections import deque

def topological_sort_kahn(V, adj):
    """Kahn's Algorithm (BFS-based)"""
    in_degree = [0] * V

    # Compute in-degrees
    for u in range(V):
        for v in adj[u]:
            in_degree[v] += 1

    # Queue for vertices with in-degree 0
    q = deque([i for i in range(V) if in_degree[i] == 0])

    result = []
    while q:
        u = q.popleft()
        result.append(u)

        for v in adj[u]:
            in_degree[v] -= 1
            if in_degree[v] == 0:
                q.append(v)

    # If result doesn't contain all vertices, graph has a cycle
    if len(result) != V:
        return [] # cycle detected

    return result

def topological_sort_dfs(V, adj):
    """DFS-based Topological Sort"""
    visited = [False] * V
    stack = []

```

```
def dfs(u):
    visited[u] = True
    for v in adj[u]:
        if not visited[v]:
            dfs(v)
    stack.append(u)

for i in range(V):
    if not visited[i]:
        dfs(i)

return stack[::-1] # reverse the stack

def has_cycle(V, adj):
    """Detect cycle in directed graph using Kahn's algorithm"""
    return len(topological_sort_kahn(V, adj)) != V

# Example
V = 6
adj = [[] for _ in range(V)]
adj[5] = [2, 0]
adj[4] = [0, 1]
adj[2] = [3]
adj[3] = [1]

order = topological_sort_kahn(V, adj)
print("Topological Order:", order) # [4, 5, 2, 0, 3, 1]
print("Has cycle:", has_cycle(V, adj)) # False
```

10. Code Explanation

- **Kahn's Algorithm:**
 - **In-degree computation:** Count incoming edges for each vertex.
 - **Queue initialization:** All vertices with in-degree 0 have no prerequisites.
 - **Processing loop:** Remove a vertex, add to result, decrement in-degree of neighbors.
 - **Cycle detection:** If `result.size() != V`, there's a cycle.
- **DFS-based Algorithm:**
 - **DFS traversal:** Visit all vertices reachable from the current vertex.
 - **Post-order stack:** After exploring all neighbors, push the current vertex to the stack.
 - **Reverse:** The stack reversed gives the topological order.
- **Longest Path in DAG:**
 - Process vertices in topological order.
 - For each vertex, relax its outgoing edges (update distances to neighbors).
 - Since we process in topological order, all dependencies are processed before their dependents.

11. Complexity Analysis

Operation	Complexity
Kahn's Algorithm	$O(V + E)$
DFS-based Topological	$O(V + E)$
Cycle Detection	$O(V + E)$
Longest Path in DAG	$O(V + E)$
Space	$O(V)$

12. Common Patterns

Course Schedule (Prerequisite Checking)

- **How to identify:** "Can you finish all courses given prerequisites?"
- **Approach:** Topological sort / cycle detection.
- **Example:** LeetCode 207 "Course Schedule"

Course Schedule II (Order of Courses)

- **How to identify:** "Return the order of courses to take."
- **Approach:** Kahn's algorithm to get the topological order.
- **Example:** LeetCode 210 "Course Schedule II"

Alien Dictionary

- **How to identify:** "Given a sorted dictionary of an alien language, find the order of characters."
- **Approach:** Build a graph from adjacent words, topological sort.
- **Example:** LeetCode 269 "Alien Dictionary"

Longest Path in a DAG

- **How to identify:** "Find the longest path in a DAG."
- **Approach:** Topological sort + DP relaxation.
- **Example:** "Critical path in a project"

Minimum Height Trees

- **How to identify:** "Find roots that minimize tree height."
- **Approach:** Kahn's algorithm (topological removal of leaves).
- **Example:** LeetCode 310 "Minimum Height Trees"

13. Common Mistakes

- **Not handling cycles:** If the graph has a cycle, topological sort is not defined. Always check if the result contains all vertices.
- **Wrong direction of edges:** Make sure edges go from prerequisite to dependent ($u \rightarrow v$ means u must come before v).

- **Forgetting Kahn's algorithm needs a queue:** Using a stack (DFS) instead of a queue gives a different valid order, but both work.
- **Incorrect in-degree decrement:** Only decrement in-degree for neighbors of the removed vertex.
- **DFS stack overflow:** For very deep graphs, recursion limit may be exceeded. Use Kahn's (iterative) or increase recursion limit.

14. Edge Cases

- **Empty graph ($V = 0$):** Return empty result.
- **Single vertex:** Return [0].
- **Disconnected DAG:** Multiple components, all valid topological orders.
- **Graph with cycle:** Return empty result (or throw an error).
- **Graph with multiple valid orders:** Both Kahn's and DFS can produce different valid orders.

15. Variations

Lexicographically Smallest Topological Order

- Use a min-heap (priority queue) instead of a queue in Kahn's algorithm.
- Ensures the smallest vertex is processed first when multiple are available.

Topological Sort with Constraints

- Some vertices must appear before others (additional constraints).
- Can be handled by adding edges or using a custom comparator.

Parallel Topological Sort

- Process vertices with in-degree 0 in parallel.
- Used in build systems (make, Bazel) to parallelize compilation.

16. Related Algorithms/Data Structures

- **DFS:** The DFS-based topological sort is a direct application of DFS.
- **BFS:** Kahn's algorithm is BFS-based.
- **Cycle Detection:** Kahn's algorithm naturally detects cycles.
- **Strongly Connected Components (SCC):** Kosaraju's algorithm uses topological sort on the condensation graph.
- **Shortest Path in DAG:** Topological sort + DP gives shortest/longest path in $O(V+E)$.

17. Practice Problems

Easy

- **Course Schedule** — LeetCode 207 (cycle detection in DAG)
- **Find Eventual Safe States** — LeetCode 802 (reverse topological sort)

Medium

- **Course Schedule II** — LeetCode 210 (return topological order)

- **Alien Dictionary** — LeetCode 269 (build graph from dictionary, topological sort)
- **Minimum Height Trees** — LeetCode 310 (topological removal of leaves)

Hard

- **Parallel Courses** — LeetCode 1136 (minimum semesters to take all courses)
- **Sorting Items by Groups Respecting Dependencies** — LeetCode 1203 (two-level topological sort)

18. Interview Explanation

"Topological sort gives a linear ordering of vertices in a DAG such that for every edge $u \rightarrow v$, u comes before v . I use Kahn's algorithm, which computes in-degrees, then repeatedly removes vertices with in-degree 0. If the result doesn't contain all vertices, the graph has a cycle. It runs in $O(V+E)$ time and $O(V)$ space. It's used for course scheduling, dependency resolution, and finding the longest path in a DAG."

19. Revision Notes

- Only for DAGs (directed acyclic graphs).
- Kahn's: queue of in-degree 0 vertices, remove, decrement neighbors.
- Cycle detection: if `result.size() != V`, cycle exists.
- DFS: post-order push to stack, reverse.
- $O(V+E)$ time, $O(V)$ space.
- Longest path in DAG: topological sort + DP.

20. Final Cheat Sheet

Property	Value
When to use	DAG ordering, dependency resolution, cycle detection
Algorithm	Kahn's (BFS) or DFS
Time	$O(V + E)$
Space	$O(V)$
Key code idea	<code>in-degree[v]++; if (--in-degree[v] == 0) q.push(v);</code>
Edge cases	Empty, single, connected, disconnected, cyclic

13. SORT + GREEDY

1. Overview

Sort + Greedy is a powerful problem-solving pattern where you sort the input first, then apply a greedy algorithm to solve the problem. Sorting organizes the data so that the greedy choice (taking the locally optimal option) leads to a globally optimal solution.

2. Intuition

Simple explanation: Many problems become easy if you process elements in a specific order. Sorting puts elements in the right order (e.g., smallest first, earliest deadline first), and then the greedy algorithm makes the best choice at each step.

Analogy: Packing suitcases for a trip. You have items of different sizes, and you want to maximize what you take. If you sort items by value/weight ratio and pack the most valuable ones first, you're doing a greedy algorithm on sorted data.

Step-by-step reasoning:

1. Identify the order in which processing elements gives the optimal solution.
2. Sort the input by that order.
3. Iterate through the sorted elements, making the greedy choice at each step.
4. The result is globally optimal (for problems with the greedy-choice property).

Why it works: For problems with the greedy-choice property (a locally optimal choice leads to a globally optimal solution), sorting ensures we make the best choices first. The optimal substructure property ensures that the remaining subproblem can be solved independently.

3. When to Use It

- When the problem has the greedy-choice property.
- When the problem has optimal substructure.
- When you need to maximize or minimize some quantity with constraints.
- When the problem involves intervals, schedules, or deadlines.
- When the problem involves assigning resources to tasks.

Trigger phrases:

- "Maximum number of ..."
- "Minimum number of ..."
- "Schedule"
- "Assign"
- "Weights and values"
- "Deadline"
- "Fractional knapsack"
- "Activity selection"

4. When Not to Use It

- When the problem doesn't have the greedy-choice property (greedy fails).
- When the problem requires DP (e.g., 0/1 knapsack, where greedy doesn't work).
- When the optimal solution requires looking ahead (greedy is myopic).
- When you need all solutions, not just one.

Simpler alternatives: Dynamic Programming (when greedy fails), Brute Force (for small n).

5. Core Concepts

Greedy-Choice Property

A globally optimal solution can be arrived at by making a locally optimal (greedy) choice.

Optimal Substructure

An optimal solution to the problem contains optimal solutions to subproblems.

Exchange Argument

A proof technique for greedy algorithms: show that any optimal solution can be transformed into the greedy solution without decreasing optimality.

Sorting by Different Keys

The key insight is knowing *what* to sort by:

- **Activity Selection:** Sort by end time.
- **Fractional Knapsack:** Sort by value/weight ratio.
- **Job Sequencing:** Sort by profit.
- **Minimum Platforms:** Sort by start and end times.
- **Huffman Coding:** Sort by frequency.

6. Step-by-Step Algorithm (Activity Selection)

Input: intervals $[(s_1, e_1), (s_2, e_2), \dots, (s_n, e_n)]$

```

1. Sort intervals by end time.
2. count = 1, lastEnd = intervals[0].end
3. For i = 1 to n-1:
    If intervals[i].start >= lastEnd:
        count++
        lastEnd = intervals[i].end
4. Return count

```

7. Dry Run

Problem: Maximum number of non-overlapping meetings. **Input:** $[(1, 3), (2, 4), (3, 5), (4, 6), (5, 7)]$

Sort by end time: Already sorted.

i	Interval	start >= lastEnd?	count	lastEnd
0	(1,3)	-	1	3
1	(2,4)	2 >= 3? No	1	3
2	(3,5)	3 >= 3? Yes	2	5
3	(4,6)	4 >= 5? No	2	5
4	(5,7)	5 >= 5? Yes	3	7

Result: 3 meetings (1-3, 3-5, 5-7)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Activity Selection (Maximum Non-Overlapping Intervals)
int activitySelection(vector<pair<int,int>>& intervals) {
    if (intervals.empty()) return 0;

    // Sort by end time
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
        return a.second < b.second;
    });

    int count = 1;
    int lastEnd = intervals[0].second;

    for (int i = 1; i < intervals.size(); ++i) {
        if (intervals[i].first >= lastEnd) {
            count++;
            lastEnd = intervals[i].second;
        }
    }
    return count;
}

// Fractional Knapsack
double fractionalKnapsack(vector<pair<int,int>>& items, int capacity) {
    // items[i] = {value, weight}
    // Sort by value/weight ratio descending
    sort(items.begin(), items.end(), [](auto& a, auto& b) {
        double ratioA = (double)a.first / a.second;
        double ratioB = (double)b.first / b.second;
        return ratioA > ratioB;
    });

    double totalValue = 0.0;
    int remaining = capacity;

    for (auto& [value, weight] : items) {
        if (weight <= remaining) {
            totalValue += value;
            remaining -= weight;
        } else {
            totalValue += (double)value * remaining / weight;
            break;
        }
    }
    return totalValue;
}
```

```

// Job Sequencing with Deadlines
struct Job {
    int id, deadline, profit;
};

vector<int> jobSequencing(vector<Job>& jobs) {
    // Sort by profit descending
    sort(jobs.begin(), jobs.end(), [](auto& a, auto& b) {
        return a.profit > b.profit;
    });

    int maxDeadline = 0;
    for (auto& job : jobs) maxDeadline = max(maxDeadline, job.deadline);

    vector<int> slot(maxDeadline + 1, -1); // slots indexed by time
    int totalProfit = 0;
    int count = 0;

    for (auto& job : jobs) {
        // Find a free slot before the deadline
        for (int t = job.deadline; t >= 1; --t) {
            if (slot[t] == -1) {
                slot[t] = job.id;
                totalProfit += job.profit;
                count++;
                break;
            }
        }
    }
    return {count, totalProfit};
}

// Minimum Number of Platforms Required
int minPlatforms(vector<pair<int,int>>& trains) {
    // trains[i] = {arrival, departure}
    vector<int> arrivals, departures;
    for (auto& [a, d] : trains) {
        arrivals.push_back(a);
        departures.push_back(d);
    }

    sort(arrivals.begin(), arrivals.end());
    sort(departures.begin(), departures.end());

    int platforms = 0, maxPlatforms = 0;
    int i = 0, j = 0;

    while (i < arrivals.size()) {
        if (arrivals[i] <= departures[j]) {
            platforms++;
            maxPlatforms = max(maxPlatforms, platforms);
            i++;
        } else {

```

```

        platforms--;
        j++;
    }
}
return maxPlatforms;
}

// Huffman Coding (basic structure)
struct Node {
    char ch;
    int freq;
    Node *left, *right;
    Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
};

struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

Node* huffmanCoding(vector<pair<char,int>>& chars) {
    priority_queue<Node*, vector<Node*>, Compare> pq;
    for (auto& [ch, freq] : chars) {
        pq.push(new Node(ch, freq));
    }

    while (pq.size() > 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();
        Node* internal = new Node('\0', left->freq + right->freq);
        internal->left = left;
        internal->right = right;
        pq.push(internal);
    }
    return pq.top(); // root of Huffman tree
}

int main() {
    // Activity Selection
    vector<pair<int,int>> intervals = {{1,3}, {2,4}, {3,5}, {4,6}, {5,7}};
    cout << "Max activities: " << activitySelection(intervals) << endl;
    // Output: 3

    // Fractional Knapsack
    vector<pair<int,int>> items = {{60, 10}, {100, 20}, {120, 30}};
    cout << "Max value: " << fractionalKnapsack(items, 50) << endl;
    // Output: 240

    return 0;
}

```

9. Python Implementation

```
def activity_selection(intervals):
    """Maximum number of non-overlapping intervals"""
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1]) # sort by end time
    count = 1
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= last_end:
            count += 1
            last_end = end

    return count

def fractional_knapsack(items, capacity):
    """items = [(value, weight), ...], maximize value"""
    items.sort(key=lambda x: x[0] / x[1], reverse=True) # sort by value/weight

    total_value = 0.0
    remaining = capacity

    for value, weight in items:
        if weight <= remaining:
            total_value += value
            remaining -= weight
        else:
            total_value += value * remaining / weight
            break

    return total_value

def job_sequencing(jobs):
    """jobs = [(id, deadline, profit), ...]"""
    jobs.sort(key=lambda x: x[2], reverse=True) # sort by profit descending

    max_deadline = max(job[1] for job in jobs)
    slot = [-1] * (max_deadline + 1)

    total_profit = 0
    count = 0

    for job_id, deadline, profit in jobs:
        for t in range(deadline, 0, -1):
            if slot[t] == -1:
                slot[t] = job_id
                total_profit += profit
                count += 1
                break
```

```

    return count, total_profit

def min_platforms(trains):
    """trains = [(arrival, departure), ...]"""
    arrivals = sorted([a for a, d in trains])
    departures = sorted([d for a, d in trains])

    platforms = 0
    max_platforms = 0
    i = j = 0

    while i < len(arrivals):
        if arrivals[i] <= departures[j]:
            platforms += 1
            max_platforms = max(max_platforms, platforms)
            i += 1
        else:
            platforms -= 1
            j += 1

    return max_platforms

# Example
intervals = [(1, 3), (2, 4), (3, 5), (4, 6), (5, 7)]
print("Max activities:", activity_selection(intervals)) # 3

```

10. Code Explanation

- **Activity Selection:** Sort by end time, greedily pick the earliest-finishing interval that doesn't overlap.
- **Fractional Knapsack:** Sort by value/weight ratio, take as much as possible of the most valuable item.
- **Job Sequencing:** Sort by profit, schedule each job at the latest available slot before its deadline.
- **Minimum Platforms:** Sort arrivals and departures separately, use two pointers to simulate the timeline.
- **Huffman Coding:** Build a min-heap of frequencies, repeatedly merge the two smallest nodes.

11. Complexity Analysis

Problem	Complexity
Activity Selection	$O(n \log n)$
Fractional Knapsack	$O(n \log n)$
Job Sequencing	$O(n \log n + n \cdot d)$ where $d = \text{max deadline}$
Minimum Platforms	$O(n \log n)$
Huffman Coding	$O(n \log n)$

12. Common Patterns

Interval Scheduling (Activity Selection)

- **How to identify:** Maximum number of non-overlapping intervals.
- **Approach:** Sort by end time, greedily pick.
- **Example:** LeetCode 435 "Non-overlapping Intervals"

Fractional Knapsack

- **How to identify:** Maximize value with weight constraint, items can be taken fractionally.
- **Approach:** Sort by value/weight ratio.
- **Example:** GFG "Fractional Knapsack"

Job Sequencing with Deadlines

- **How to identify:** Maximize profit from jobs with deadlines.
- **Approach:** Sort by profit, schedule at the latest free slot.
- **Example:** GFG "Job Sequencing Problem"

Minimum Platforms / Meeting Rooms II

- **How to identify:** Minimum resources needed for overlapping intervals.
- **Approach:** Sort starts and ends separately, sweep line.
- **Example:** LeetCode 253 "Meeting Rooms II"

Huffman Coding

- **How to identify:** Minimum encoding length for characters.
- **Approach:** Build a binary tree using a min-heap.
- **Example:** GFG "Huffman Coding"

13. Common Mistakes

- **Wrong sorting order:** Activity selection needs end time sorting, not start time.
- **Greedy doesn't work for 0/1 knapsack:** Fractional knapsack works with greedy, but 0/1 knapsack needs DP.
- **Not considering ties:** When multiple elements have the same key, the secondary criterion matters.
- **Overlapping condition:** For activity selection, `start >= lastEnd` (not `>`). If `>=`, intervals can touch.
- **Job scheduling: not checking all slots:** Always check from the latest to the earliest slot.

14. Edge Cases

- **Empty input:** Return 0 or empty.
- **Single element:** Always feasible.
- **All overlapping:** Only 1 activity can be selected.
- **No overlapping:** All activities can be selected.
- **Capacity smaller than any item:** Fractional knapsack still works (take a fraction).
- **All deadlines are 0:** No job can be scheduled.

15. Variations

Weighted Interval Scheduling

- Each interval has a weight, maximize total weight of non-overlapping intervals.
- Requires DP, not greedy (greedy doesn't work when weights are not uniform).

0/1 Knapsack

- Items cannot be taken fractionally.
- Requires DP (not greedy).

Greedy on Trees (Prim's, Kruskal's)

- Minimum spanning tree algorithms.
- Sort edges by weight, greedily add edges that don't form cycles.

16. Related Algorithms/Data Structures

- **Dynamic Programming:** When greedy fails, DP is the alternative.
- **Sorting:** The foundation of the Sort + Greedy pattern.
- **Priority Queue:** Often used in greedy algorithms to maintain the best element dynamically.
- **Interval Tree:** For complex interval queries.

17. Practice Problems

Easy

- **Assign Cookies** — LeetCode 455 (sort children and cookies, greedy)
- **Minimum Number of Arrows to Burst Balloons** — LeetCode 452 (sort by end, greedy)

Medium

- **Non-overlapping Intervals** — LeetCode 435 (activity selection)
- **Meeting Rooms II** — LeetCode 253 (minimum platforms)
- **Job Sequencing Problem** — GFG (deadline-based scheduling)

Hard

- **Maximum Profit in Job Scheduling** — LeetCode 1235 (weighted interval scheduling, DP + binary search)
- **Minimum Number of Taps to Open to Water a Garden** — LeetCode 1326 (interval covering, greedy)

18. Interview Explanation

"The Sort + Greedy pattern is one of the most common in competitive programming. You sort the input by a specific key, then make locally optimal choices that lead to a globally optimal solution. Classic examples include Activity Selection (sort by end time), Fractional Knapsack (sort by value/weight ratio), and Job Sequencing (sort by profit). The key is identifying the correct sorting order and proving that the greedy choice is optimal, usually via an exchange argument."

19. Revision Notes

- Sort by the right key (end time, ratio, profit, etc.).

- Greedy: make the best local choice.
- Works for problems with greedy-choice property + optimal substructure.
- Common: activity selection, fractional knapsack, job sequencing, min platforms.
- $O(n \log n) + O(n)$ for the greedy pass.

20. Final Cheat Sheet

Property	Value
When to use	Maximize/minimize with constraints, scheduling, resource allocation
Steps	1. Sort by key 2. Greedy pass
Common keys	End time, value/weight, profit, deadline
Time	$O(n \log n)$
Key code idea	<code>sort(...); for (x : sorted) { if (condition) { take x; update; } }</code>
Edge cases	Empty, single, all overlap, no overlap, zero capacity

14. EXTERNAL SORTING

1. Overview

External Sorting is a class of sorting algorithms that handle massive amounts of data — too large to fit entirely in the computer's main memory (RAM). It uses disk storage to store intermediate results and typically uses a divide-and-conquer approach based on Merge Sort.

2. Intuition

Simple explanation: When data is too large to fit in RAM, you can't use normal sorting algorithms. External sorting splits the data into chunks that do fit in RAM, sorts each chunk in memory, writes them to disk, and then merges the sorted chunks together.

Analogy: Imagine sorting a million books on a table that can only hold 100 books. You sort them in groups of 100, put each sorted group back on shelves, then merge the sorted groups by taking the smallest book from each group's shelf.

Step-by-step reasoning:

1. **Split phase:** Divide the large file into smaller chunks that fit in RAM.
2. **Sort phase:** Sort each chunk in memory (using Quick Sort, Merge Sort, etc.), write each sorted chunk to a temporary file on disk.
3. **Merge phase:** Merge the sorted chunks together using a k-way merge (using a min-heap).
4. **Output:** Write the final sorted data to the output file.

Why it works: The sort phase is just standard sorting on small enough data. The merge phase is the same as merging sorted arrays, but using disk I/O. The k-way merge with a min-heap ensures we only keep one element from each chunk in memory at a time.

3. When to Use It

- When data is too large to fit in RAM (e.g., sorting a 100GB file on a machine with 8GB RAM).
- When sorting database tables that don't fit in memory.
- When processing large log files.
- When dealing with big data applications.
- When the data is stored on disk and cannot be loaded entirely into memory.

Trigger phrases:

- "Too large to fit in memory"
- "External sort"
- "Sort large file"
- "Database sorting"
- "K-way merge"
- "Big data sorting"

4. When Not to Use It

- When the data fits in RAM (internal sorting is much faster).
- When the data is small enough for `sort()` in memory.
- When you need real-time sorting (external sorting has high latency due to disk I/O).
- When you have limited disk space (external sorting needs temporary disk space for intermediate files).

Simpler alternatives: `sort()` in standard library, `std::sort`, Python's `sort()`, Unix `sort` command (which handles external sorting).

5. Core Concepts

Run (Chunk)

A sorted chunk of data that fits in memory and is written to a temporary file.

K-Way Merge

Merging k sorted runs simultaneously using a min-heap of size k.

Page / Block

The unit of disk I/O. Reading/writing in blocks is more efficient than reading element by element.

Memory-External Tradeoff

More memory → larger runs → fewer runs → fewer merge passes → faster sorting.

Merge Pass

One pass of merging. If we have k runs and can merge M at a time, we need $\text{ceil}(\log_M(k))$ merge passes.

6. Step-by-Step Algorithm

Input: Large file on disk, memory limit M

1. Sort Phase:
 - a. Read M elements from the input file into memory.
 - b. Sort them using an in-memory sort (e.g., Quick Sort).
 - c. Write the sorted chunk (run) to a temporary file.
 - d. Repeat until all input is processed.
2. Merge Phase (K-way Merge):
 - a. Open all temporary files for reading.
 - b. Create a min-heap of size = number of runs.
 - c. Read one element from each run into the heap.
 - d. While heap is not empty:
 - Extract the minimum element from the heap.
 - Write it to the output file.
 - Read the next element from the run that provided the minimum.
 - If the run is not exhausted, push the new element into the heap.
 - e. Close all temporary files and delete them.

7. Dry Run

Input: Large file with numbers: [38, 27, 43, 3, 9, 82, 10, 45, 6, 15, 33, 71] **Memory limit:** Can hold 4 elements at a time.

Sort Phase (run size = 4):

Run	Elements in Memory	Sorted	Written to Temp File
1	[38, 27, 43, 3]	[3, 27, 38, 43]	run1.txt
2	[9, 82, 10, 45]	[9, 10, 45, 82]	run2.txt
3	[6, 15, 33, 71]	[6, 15, 33, 71]	run3.txt

Merge Phase (3-way merge):

Step	Heap (value, run)	Extract	Output
1	(3,1), (9,2), (6,3)	3 (run1)	[3]
2	(27,1), (9,2), (6,3)	6 (run3)	[3, 6]
3	(27,1), (9,2), (15,3)	9 (run2)	[3, 6, 9]
4	(27,1), (10,2), (15,3)	10 (run2)	[3, 6, 9, 10]
5	(27,1), (45,2), (15,3)	15 (run3)	[3, 6, 9, 10, 15]
6	(27,1), (45,2), (33,3)	27 (run1)	[3, 6, 9, 10, 15, 27]
7	(38,1), (45,2), (33,3)	33 (run3)	[3, 6, 9, 10, 15, 27, 33]
8	(38,1), (45,2), (71,3)	38 (run1)	[3, 6, 9, 10, 15, 27, 33, 38]

Step	Heap (value, run)	Extract	Output
9	(43,1), (45,2), (71,3)	43 (run1)	[3, 6, 9, 10, 15, 27, 33, 38, 43]
10	- (run1 done), (45,2), (71,3)	45 (run2)	[3, 6, 9, 10, 15, 27, 33, 38, 43, 45]
11	(82,2), (71,3)	71 (run3)	[3, 6, 9, 10, 15, 27, 33, 38, 43, 45, 71]
12	(82,2)	82 (run2)	[3, 6, 9, 10, 15, 27, 33, 38, 43, 45, 71, 82]

Final: [3, 6, 9, 10, 15, 27, 33, 38, 43, 45, 71, 82]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// External sorting simulation (in-memory for illustration)
// Real external sorting would use file I/O

const int RUN_SIZE = 4; // number of elements per run (typically millions)

// Generate sorted runs (simulated)
vector<vector<int>> generateRuns(const vector<int>& data) {
    vector<vector<int>> runs;
    for (int i = 0; i < data.size(); i += RUN_SIZE) {
        vector<int> run(data.begin() + i,
                        data.begin() + min((int)data.size(), i + RUN_SIZE));
        sort(run.begin(), run.end());
        runs.push_back(run);
        cout << "Run " << runs.size() << ": ";
        for (int x : run) cout << x << " ";
        cout << endl;
    }
    return runs;
}

// K-way merge using a min-heap
// Each element in the heap is a pair (value, run_index)
vector<int> kWayMerge(const vector<vector<int>>& runs) {
    // Min-heap: pair(value, index_of_run)
    // Also need to track the position within each run
    using HeapNode = pair<int, pair<int, int>>; // (value, (run_idx, pos_in_run))

    auto cmp = [](const HeapNode& a, const HeapNode& b) {
        return a.first > b.first; // min-heap
    };
    priority_queue<HeapNode, vector<HeapNode>, decltype(cmp)> pq(cmp);

    vector<int> positions(runs.size(), 0); // current position in each run

    // Initialize heap with first element from each run
    for (int i = 0; i < runs.size(); ++i) {
```

```

        if (!runs[i].empty()) {
            pq.push({runs[i][0], {i, 0}});
            positions[i] = 1;
        }
    }

    vector<int> result;
    while (!pq.empty()) {
        auto [value, info] = pq.top();
        pq.pop();

        int runIdx = info.first;
        int pos = info.second;

        result.push_back(value);

        // Add next element from the same run (if any)
        if (positions[runIdx] < runs[runIdx].size()) {
            int nextPos = positions[runIdx]++;
            pq.push({runs[runIdx][nextPos], {runIdx, nextPos}});
        }
    }

    return result;
}

// Simulated external sort
vector<int> externalSort(const vector<int>& data) {
    cout << "=== Sort Phase ===" << endl;
    auto runs = generateRuns(data);

    cout << "\n=== Merge Phase ===" << endl;
    auto result = kWayMerge(runs);

    return result;
}

// Real-world external sort with file I/O (conceptual)
class ExternalSorter {
private:
    string inputFile, outputFile;
    size_t memoryLimit; // in bytes

    // Sort a chunk of data and write to a temp file
    string sortAndWriteChunk(const vector<int>& chunk, int chunkNum) {
        string tempFile = "temp_run_" + to_string(chunkNum) + ".txt";
        vector<int> sorted = chunk;
        sort(sorted.begin(), sorted.end());

        ofstream out(tempFile);
        for (int x : sorted) out << x << "\n";
        out.close();

        return tempFile;
    }
};

```

```

    }

    // Merge sorted temp files into output
    void mergeFiles(const vector<string>& tempFiles) {
        // Open all temp files
        vector<ifstream> inputs;
        for (const auto& f : tempFiles) {
            inputs.emplace_back(f);
        }

        // Min-heap: (value, file_index)
        using HeapNode = pair<int, int>;
        auto cmp = [](const HeapNode& a, const HeapNode& b) {
            return a.first > b.first;
        };
        priority_queue<HeapNode, vector<HeapNode>, decltype(cmp)> pq(cmp);

        ofstream out(outputFile);

        // Initialize heap with first element from each file
        vector<int> buffer(tempFiles.size());
        for (int i = 0; i < tempFiles.size(); ++i) {
            if (inputs[i] >> buffer[i]) {
                pq.push({buffer[i], i});
            }
        }

        while (!pq.empty()) {
            auto [value, idx] = pq.top();
            pq.pop();
            out << value << "\n";

            if (inputs[idx] >> buffer[idx]) {
                pq.push({buffer[idx], idx});
            }
        }

        out.close();
        for (auto& in : inputs) in.close();

        // Clean up temp files
        for (const auto& f : tempFiles) {
            remove(f.c_str());
        }
    }

public:
    ExternalSorter(const string& inFile, const string& outFile, size_t memLimit)
        : inputFile(inFile), outputFile(outFile), memoryLimit(memLimit) {}

    void sort() {
        ifstream in(inputFile);
        vector<int> chunk;
        int value;
    }

```

```

    int chunkNum = 0;
    vector<string> tempFiles;

    // Sort Phase
    while (in >> value) {
        chunk.push_back(value);
        // Estimate: sizeof(int) * chunk.size() >= memoryLimit
        if (chunk.size() * sizeof(int) >= memoryLimit) {
            string tempFile = sortAndWriteChunk(chunk, chunkNum++);
            tempFiles.push_back(tempFile);
            chunk.clear();
        }
    }
    // Last chunk
    if (!chunk.empty()) {
        string tempFile = sortAndWriteChunk(chunk, chunkNum++);
        tempFiles.push_back(tempFile);
    }
    in.close();

    // Merge Phase
    mergeFiles(tempFiles);
}

};

int main() {
    // Simulated external sort
    vector<int> data = {38, 27, 43, 3, 9, 82, 10, 45, 6, 15, 33, 71};
    auto sorted = externalSort(data);

    cout << "\nSorted: ";
    for (int x : sorted) cout << x << " ";
    cout << endl;
    // Output: 3 6 9 10 15 27 33 38 43 45 71 82

    return 0;
}

```

9. Python Implementation

```

import heapq
import os
import tempfile

RUN_SIZE = 4 # elements per run (for simulation)

def generate_runs(data):
    """Generate sorted runs (simulated)"""
    runs = []
    for i in range(0, len(data), RUN_SIZE):
        run = sorted(data[i:min(i + RUN_SIZE, len(data))])

```

```

        runs.append(run)
        print(f"Run {len(runs)}: {run}")
    return runs

def k_way_merge(runs):
    """K-way merge using a min-heap"""
    heap = []
    positions = [0] * len(runs)

    # Initialize heap with first element from each run
    for i, run in enumerate(runs):
        if run:
            heapq.heappush(heap, (run[0], i))
            positions[i] = 1

    result = []
    while heap:
        value, run_idx = heapq.heappop(heap)
        result.append(value)

        # Add next element from the same run
        if positions[run_idx] < len(runs[run_idx]):
            next_pos = positions[run_idx]
            positions[run_idx] += 1
            heapq.heappush(heap, (runs[run_idx][next_pos], run_idx))

    return result

def external_sort_simulated(data):
    """Simulated external sort"""
    print("=== Sort Phase ===")
    runs = generate_runs(data)
    print("\n=== Merge Phase ===")
    result = k_way_merge(runs)
    return result

# Real-world external sort with file I/O
def external_sort_file(input_file, output_file, memory_limit_bytes):
    """
    Sort a large file using external sorting.
    """
    temp_files = []

    # Sort Phase: split into chunks, sort each, write to disk
    with open(input_file, 'r') as f:
        chunk = []
        chunk_size = 0
        chunk_num = 0

        for line in f:
            value = int(line.strip())
            chunk.append(value)
            chunk_size += 4 # sizeof(int) ≈ 4 bytes

```

```

        if chunk_size >= memory_limit_bytes:
            chunk.sort()
            temp_file = f"temp_run_{chunk_num}.txt"
            with open(temp_file, 'w') as tf:
                for v in chunk:
                    tf.write(f"{v}\n")
            temp_files.append(temp_file)
            chunk = []
            chunk_size = 0
            chunk_num += 1

# Last chunk
if chunk:
    chunk.sort()
    temp_file = f"temp_run_{chunk_num}.txt"
    with open(temp_file, 'w') as tf:
        for v in chunk:
            tf.write(f"{v}\n")
    temp_files.append(temp_file)

# Merge Phase: k-way merge
with open(output_file, 'w') as out:
    # Open all temp files
    inputs = [open(f, 'r') for f in temp_files]

    # Initialize heap
    heap = []
    for i, f in enumerate(inputs):
        line = f.readline()
        if line:
            heapq.heappush(heap, (int(line.strip()), i))

    # K-way merge
    while heap:
        value, idx = heapq.heappop(heap)
        out.write(f"{value}\n")

        line = inputs[idx].readline()
        if line:
            heapq.heappush(heap, (int(line.strip()), idx))

    # Close all files
    for f in inputs:
        f.close()

# Clean up temp files
for f in temp_files:
    os.remove(f)

# Example with simulated data
data = [38, 27, 43, 3, 9, 82, 10, 45, 6, 15, 33, 71]
sorted_data = external_sort_simulated(data)

```

```
print(f"\nSorted: {sorted_data}")
# [3, 6, 9, 10, 15, 27, 33, 38, 43, 45, 71, 82]
```

10. Code Explanation

- **Sort Phase:** Reads chunks of data that fit in memory, sorts them with an in-memory sort, and writes each sorted chunk (run) to a temporary file.
- **K-way Merge:** Uses a min-heap to merge k sorted runs simultaneously. The heap always contains the smallest remaining element from each run.
- **Heap initialization:** First element from each run is pushed into the heap.
- **Merge loop:** Extract the minimum from the heap, write to output, and push the next element from the same run.
- **Memory management:** The heap size is k (number of runs), which is much smaller than the data size.
- **File I/O:** Reading and writing in blocks optimizes disk access.

11. Complexity Analysis

Operation	Complexity
Sort Phase (per chunk)	$O((n/M) \cdot M \log M) = O(n \log M)$
Merge Phase	$O(n \log k)$ where k = number of runs
Total Time	$O(n \log M + n \log k) = O(n \log n)$
Disk I/O	$O(n)$ reads + $O(n)$ writes per pass
Space (RAM)	$O(M)$ for sorting + $O(k)$ for heap

Where: M = memory limit (in elements), k = number of runs = $\text{ceil}(n/M)$

12. Common Patterns

Sorting Large Files

- **How to identify:** "Sort a file that doesn't fit in memory."
- **Approach:** External Merge Sort.
- **Example:** "Sort a 100GB log file"

Database External Sort

- **How to identify:** "Sorting a database table."
- **Approach:** External sort with B-tree indexing.
- **Example:** "ORDER BY clause in SQL"

Multi-pass Merge

- **How to identify:** Too many runs to merge at once.
- **Approach:** Merge runs in passes (merge M runs at a time, produce larger runs, repeat).
- **Example:** "Sorting with very limited memory"

13. Common Mistakes

- **Ignoring disk I/O costs:** Disk access is thousands of times slower than RAM. Minimize the number of passes.
- **Not using buffered I/O:** Reading/writing one element at a time is extremely slow. Use blocks.
- **Too many temporary files:** Each temp file consumes file descriptors. Manage them carefully.
- **Not cleaning up temp files:** Temporary files should be deleted after the merge.
- **Incorrect memory calculation:** The memory limit must account for the heap, buffers, and other overhead.

14. Edge Cases

- **Data fits in memory:** External sorting is overkill. Use internal sorting.
- **Single run:** The merge phase is a no-op (just copy the run to output).
- **Empty file:** No input, no output.
- **File with one element:** Trivially sorted.
- **Duplicate values:** Handled correctly by the merge.
- **Very large number of runs:** May need multiple merge passes.

15. Variations

Multi-pass External Merge Sort

- If the number of runs exceeds the available file descriptors, merge in multiple passes.
- Pass 1: Sort runs into larger runs.
- Pass 2: Merge larger runs into even larger runs.
- Continue until only one run remains.

Replacement Selection Sort

- Generates larger runs than the standard sort phase.
- Uses a heap to produce runs of size $\sim 2M$ (instead of M).
- Reduces the number of merge passes.

External Radix Sort

- For data with fixed-length keys.
- Can be more efficient than Merge Sort for certain data types.

16. Related Algorithms/Data Structures

- **Merge Sort:** The foundation of external sorting.
- **K-way Merge:** The merge phase uses k-way merge with a min-heap.
- **B-tree:** Used in databases for external sorting and indexing.
- **Priority Queue (Min-Heap):** Used in the k-way merge phase.

17. Practice Problems

Easy

- **Sort Characters By Frequency** — LeetCode 451 (bucket sort, not external)
- **Merge Two Sorted Arrays** — LeetCode 88 (practice the merge operation)

Medium

- **Merge k Sorted Lists** — LeetCode 23 (k-way merge with min-heap)
- **Find Median from Data Stream** — LeetCode 295 (two heaps, not external)

Hard

- **Design a File Sorting System** — System Design (external sorting for large files)
- **External Sort Implementation** — GFG (implement external sort with file I/O)

18. Interview Explanation

"External sorting is used when data doesn't fit in memory. It works in two phases: First, we split the data into chunks that fit in RAM, sort each chunk, and write them to temporary files. Second, we perform a k-way merge using a min-heap, reading one element from each chunk at a time. The heap always contains the smallest element from each sorted chunk, and we repeatedly extract the minimum and write it to the output. This approach has $O(n \log n)$ time complexity and uses $O(M)$ memory where M is the memory limit."

19. Revision Notes

- Two phases: Sort (create runs) + Merge (k-way merge).
- K-way merge uses a min-heap.
- Run size = memory limit M .
- Number of runs $k = \text{ceil}(n / M)$.
- Time: $O(n \log M + n \log k) = O(n \log n)$.
- Disk I/O is the bottleneck — minimize passes.

20. Final Cheat Sheet

Property	Value
When to use	Data too large for RAM
Algorithm	External Merge Sort
Phases	1. Sort runs 2. K-way merge
Time	$O(n \log n)$
Space (RAM)	$O(M)$ where M = memory limit
Key code idea	<code>sort(chunk) → write(run) → heap = min-heap of runs → while heap: extract min, write, push next</code>
Edge cases	Fits in RAM, single run, empty, duplicates

15. TIMSORT IDEA

1. Overview

TimSort is a hybrid, stable, adaptive sorting algorithm derived from Merge Sort and Insertion Sort. It was designed by Tim Peters in 2002 for Python's `sort()` method and is now used in Python, Java (for objects), JavaScript (V8), and many other languages.

2. Intuition

Simple explanation: TimSort combines the best of Insertion Sort and Merge Sort. It splits the array into small "runs" (which are either already sorted or reverse-sorted), uses Insertion Sort to sort small runs, and then merges runs using Merge Sort.

Analogy: Like cleaning a messy room. You first find small areas that are already tidy (natural runs). You tidy up small messy areas (Insertion Sort). Then you merge the tidy areas together (Merge Sort) into one big tidy room.

Step-by-step reasoning:

1. Scan the array to find natural sorted runs (increasing or decreasing sequences).
2. For small runs (below a threshold, typically 32–64), use Insertion Sort to extend them to the minimum run length.
3. Push runs onto a stack.
4. Maintain an invariant on the stack: the lengths of runs must satisfy certain properties (to keep merges balanced).
5. Merge runs from the stack (usually two adjacent runs) when the invariant is violated.
6. After all elements are processed, merge all remaining runs on the stack.

Why it works: Real-world data often has natural patterns (already sorted sequences). TimSort exploits these patterns to run in $O(n)$ time for nearly sorted data. Insertion Sort is efficient for small arrays, and Merge Sort provides stable $O(n \log n)$ worst-case behavior.

3. When to Use It

- When you need a stable sort with good performance on real-world data.
- When the data may have natural ordering (partially sorted, reverse sorted, etc.).
- When you need $O(n)$ time for nearly sorted data.
- When you need guaranteed $O(n \log n)$ worst-case time.
- When implementing a general-purpose sorting function.

Trigger phrases:

- "Hybrid sort"
- "Adaptive sort"
- "Stable sort for real-world data"
- "Python sort"
- "Java sort"
- "General purpose sort"

4. When Not to Use It

- When the data is random and large (Quick Sort is usually faster).
- When you need an in-place sort (TimSort uses $O(n)$ extra space).
- When the array is very small (Insertion Sort alone is simpler).
- When you need a non-comparison sort (use Counting Sort, Radix Sort).

Simpler alternatives: `std::sort()` (Introsort, faster for random data, but not stable), `std::stable_sort()` (Merge Sort, which is similar to TimSort but simpler).

5. Core Concepts

Natural Runs

Sequences of consecutive elements that are already in non-decreasing order (or strictly decreasing order when reversed). TimSort efficiently detects and uses these.

Minrun

A minimum run length (typically 32 to 64). If a natural run is shorter than minrun, it's extended using Insertion Sort. This ensures runs are long enough for efficient merging.

Gallop Mode

An optimization in the merge step. When one run consistently provides elements, TimSort enters "galloping mode" and uses binary search to find the correct position, reducing comparisons.

Merge Invariant

TimSort maintains an invariant on the stack of run lengths to ensure merges are balanced. The typical invariant: `stack[-3] > stack[-2] + stack[-1]` and `stack[-2] > stack[-1]`. This ensures $O(n \log n)$ behavior.

Stability

TimSort is stable because it uses Merge Sort's merge operation, which preserves the relative order of equal elements.

6. Step-by-Step Algorithm

Input: `arr[0..n-1]`

1. If $n < 2$, return.
2. Compute `minrun = optimal_run_length(n)` (typically 32-64)
3. For $i = 0$ to $n-1$:
 - a. Find a natural run starting at i (increasing or decreasing).
 - b. If the run is decreasing, reverse it.
 - c. If the run length $< \text{minrun}$:
Extend the run to minrun using Insertion Sort.
 - d. Push the run onto the stack.

- e. While the merge invariant is violated:
Merge two adjacent runs on the stack.
- f. i = end of the run.
- 4. Merge all remaining runs on the stack.

7. Dry Run

Input: [3, 7, 1, 4, 2, 8, 5, 9, 0, 6] **minrun** = 4 (for illustration)

Step 1: Find runs

Start	Scan	Natural Run	Length	minrun?	Extended Run
0	[3,7,1]	[3,7] → break at 1 ($3 > 7$, $7 > 1$)	2	<4	Insertion Sort: [3,7,1,4] → [1,3,4,7]
4	2,8,5,9,0	[2,8] → break at 5 ($2 < 8$, $8 > 5$)	2	<4	Insertion Sort: [2,8,5,9] → [2,5,8,9]
8	0,6	[0,6]	2	<4	Insertion Sort: [0,6] → can't extend (only 2 left)

Wait — let me redo with a cleaner approach.

Actually, the scan finds the longest natural run. Let me trace more carefully.

Input: [3, 7, 1, 4, 2, 8, 5, 9, 0, 6]

minrun = 4

i = 0:

- Scan: $3 \leq 7$ (increasing), $7 > 1$ (break) → natural run = [3, 7], length = 2
- $2 < 4$, so extend to minrun: include [1, 4], sort [3, 7, 1, 4] → [1, 3, 4, 7]
- Run 1: [1, 3, 4, 7], stack = [[1, 3, 4, 7]]

i = 4:

- Scan: $2 \leq 8$ (increasing), $8 > 5$ (break) → natural run = [2, 8], length = 2
- $2 < 4$, extend: include [5, 9], sort [2, 8, 5, 9] → [2, 5, 8, 9]
- Run 2: [2, 5, 8, 9], stack = [[1, 3, 4, 7], [2, 5, 8, 9]]

i = 8:

- Scan: $0 \leq 6$ (increasing) → natural run = [0, 6], length = 2
- $2 < 4$, but only 2 elements left, so just sort [0, 6] → [0, 6]
- Run 3: [0, 6], stack = [[1, 3, 4, 7], [2, 5, 8, 9], [0, 6]]

Merge phase:

- Merge [2, 5, 8, 9] and [0, 6] → [0, 2, 5, 6, 8, 9]
- Stack = [[1, 3, 4, 7], [0, 2, 5, 6, 8, 9]]

- Merge [1, 3, 4, 7] and [0, 2, 5, 6, 8, 9] → [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Final: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// TimSort is complex to implement fully. This is a simplified version.
// The real implementation has ~1000 lines in Python's source.

class TimSort {
private:
    static const int MIN_MERGE = 32;
    vector<int> arr;

    // Compute minimum run length
    static int minRunLength(int n) {
        int r = 0;
        while (n >= MIN_MERGE) {
            r |= (n & 1);
            n >>= 1;
        }
        return n + r;
    }

    // Binary insertion sort for a range
    void binaryInsertionSort(int l, int r) {
        for (int i = l + 1; i <= r; ++i) {
            int key = arr[i];
            // Binary search to find insertion position
            int pos = lower_bound(arr.begin() + l, arr.begin() + i, key) -
arr.begin();
            // Shift elements right
            for (int j = i; j > pos; --j) {
                arr[j] = arr[j - 1];
            }
            arr[pos] = key;
        }
    }

    // Merge two runs: arr[l..m] and arr[m+1..r]
    void merge(int l, int m, int r) {
        int len1 = m - l + 1, len2 = r - m;
        vector<int> left(len1), right(len2);

        for (int i = 0; i < len1; ++i) left[i] = arr[l + i];
        for (int i = 0; i < len2; ++i) right[i] = arr[m + 1 + i];

        int i = 0, j = 0, k = l;
        while (i < len1 && j < len2) {
```

```

        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }

    while (i < len1) arr[k++] = left[i++];
    while (j < len2) arr[k++] = right[j++];
}

public:
    TimSort(const vector<int>& input) : arr(input) {}

    vector<int> sort() {
        int n = arr.size();
        if (n < 2) return arr;

        int minRun = minRunLength(n);

        // Stack of runs (each run is a [start, end] pair)
        vector<pair<int, int>> runs;

        int i = 0;
        while (i < n) {
            // Find a natural run
            int start = i;
            i++;

            if (i < n) {
                // Determine if the run is increasing or decreasing
                if (arr[i] < arr[i - 1]) {
                    // Decreasing run
                    while (i < n && arr[i] < arr[i - 1]) i++;
                    // Reverse the decreasing run
                    reverse(arr.begin() + start, arr.begin() + i);
                } else {
                    // Increasing run
                    while (i < n && arr[i] >= arr[i - 1]) i++;
                }
            }

            int runLen = i - start;

            // Extend short runs to minRun
            if (runLen < minRun) {
                int end = min(start + minRun - 1, n - 1);
                binaryInsertionSort(start, end);
                i = end + 1;
                runLen = i - start;
            }

            runs.push_back({start, i - 1});

```

```

        // Merge if needed (simplified: merge when last run is too short)
        while (runs.size() >= 2) {
            auto& run2 = runs.back();
            auto& run1 = runs[runs.size() - 2];

            if (run1.second - run1.first + 1 <= run2.second - run2.first + 1)
            {
                int m = run1.second;
                int l = run1.first;
                int r = run2.second;
                merge(l, m, r);
                runs.pop_back();
                runs.pop_back();
                runs.push_back({l, r});
            } else {
                break;
            }
        }

        // Merge all remaining runs
        while (runs.size() >= 2) {
            auto run2 = runs.back(); runs.pop_back();
            auto run1 = runs.back(); runs.pop_back();
            merge(run1.first, run1.second, run2.second);
            runs.push_back({run1.first, run2.second});
        }

        return arr;
    }
};

int main() {
    vector<int> arr = {3, 7, 1, 4, 2, 8, 5, 9, 0, 6};
    TimSort sorter(arr);
    vector<int> sorted = sorter.sort();

    for (int x : sorted) cout << x << " ";
    cout << endl;
    // Output: 0 1 2 3 4 5 6 7 8 9

    return 0;
}

```

9. Python Implementation

```

# Python's built-in sort IS TimSort, so this is a simplified demo

MIN_MERGE = 32

def min_run_length(n):

```

```

"""Compute the minimum run length for TimSort"""
r = 0
while n >= MIN_MERGE:
    r |= (n & 1)
    n >>= 1
return n + r

def binary_insertion_sort(arr, l, r):
    """Sort arr[l..r] using binary insertion sort"""
    for i in range(l + 1, r + 1):
        key = arr[i]
        # Binary search for insertion position
        pos = bisect_left(arr, key, l, i)
        # Shift elements
        for j in range(i, pos, -1):
            arr[j] = arr[j - 1]
        arr[pos] = key

def merge(arr, l, m, r):
    """Merge two sorted runs arr[l..m] and arr[m+1..r]"""
    left = arr[l:m+1]
    right = arr[m+1:r+1]

    i = j = 0
    k = l

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

def timsort(arr):
    """Simplified TimSort implementation"""
    import bisect

    n = len(arr)
    if n < 2:
        return arr

    min_run = min_run_length(n)

```

```
runs = []

i = 0
while i < n:
    start = i
    i += 1

    if i < n:
        # Determine if run is increasing or decreasing
        if arr[i] < arr[i - 1]:
            # Decreasing run
            while i < n and arr[i] < arr[i - 1]:
                i += 1
            # Reverse it
            arr[start:i] = reversed(arr[start:i])
        else:
            # Increasing run
            while i < n and arr[i] >= arr[i - 1]:
                i += 1

    run_len = i - start

    # Extend short runs
    if run_len < min_run:
        end = min(start + min_run - 1, n - 1)
        binary_insertion_sort(arr, start, end)
        i = end + 1
        run_len = i - start

    runs.append((start, i - 1))

# Merge if needed (simplified invariant)
while len(runs) >= 2:
    run1 = runs[-2]
    run2 = runs[-1]
    len1 = run1[1] - run1[0] + 1
    len2 = run2[1] - run2[0] + 1

    if len1 <= len2:
        l, m = run1
        _, r = run2
        merge(arr, l, m, r)
        runs.pop()
        runs.pop()
        runs.append((l, r))
    else:
        break

# Merge all remaining runs
while len(runs) >= 2:
    run2 = runs.pop()
    run1 = runs.pop()
    merge(arr, run1[0], run1[1], run2[1])
    runs.append((run1[0], run2[1]))
```

```
        return arr

# Example
arr = [3, 7, 1, 4, 2, 8, 5, 9, 0, 6]
print(timsort(arr)) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

10. Code Explanation

- **minRunLength**: Computes the optimal minimum run length. For $n < 64$, it's n . Otherwise, it's between 32 and 64.
- **Binary Insertion Sort**: Used for sorting short runs. Binary search reduces the number of comparisons.
- **Natural run detection**: Scans to find sequences that are already sorted (or reverse sorted).
- **Run extension**: If a natural run is shorter than minrun, Insertion Sort is used to extend it.
- **Merge**: Standard merge of two sorted runs (same as Merge Sort).
- **Merge invariant**: Ensures runs on the stack are balanced for $O(n \log n)$ performance.
- **Galloping mode**: (Not implemented in the simplified version) — when one run consistently wins, it switches to binary search to find the correct position.

11. Complexity Analysis

Operation	Complexity
Time (Best)	$O(n)$
Time (Worst)	$O(n \log n)$
Time (Average)	$O(n \log n)$
Space	$O(n)$
Stable	Yes

12. Common Patterns

Why TimSort is Used in Practice

- **Real-world data is often partially sorted** — TimSort exploits this.
- **Stable sort requirement** — Many applications need stable sorting.
- **Adaptive performance** — $O(n)$ for nearly sorted data, $O(n \log n)$ worst case.

When to Choose TimSort Over Other Sorts

- When you need a general-purpose stable sort.
- When the data may have patterns (but you don't know which).
- When you're implementing a language's standard library sort.

13. Common Mistakes

- **Not understanding the merge invariant**: Without the invariant, TimSort can degrade to $O(n^2)$.
- **Incorrect run detection**: Failing to detect decreasing runs and reverse them properly.

- **Not handling the galloping mode:** The simplified version without galloping is still correct but slower.
- **Wrong minrun calculation:** Different values of minrun affect performance. The optimal is 32–64.

14. Edge Cases

- **Empty array:** Return immediately.
- **Single element:** Already sorted.
- **Already sorted:** $O(n)$ — single run, no merging needed.
- **Reverse sorted:** $O(n)$ — single decreasing run, reversed into one run.
- **All equal elements:** $O(n)$ — single run.
- **Random data:** $O(n \log n)$ — many small runs, merging needed.

15. Variations

Introsort (Used in C++ `std::sort`)

- Hybrid of Quick Sort, Heap Sort, and Insertion Sort.
- Unstable. Faster for random data than TimSort.
- Falls back to Heap Sort when recursion depth exceeds $\log n$.

Pattern-Defeating Quick Sort (PDQSort)

- A hybrid sort that combines Quick Sort and Insertion Sort.
- Used in Rust's standard library.
- Faster than TimSort for random data, but not stable.

16. Related Algorithms/Data Structures

- **Merge Sort:** The merge operation in TimSort is from Merge Sort.
- **Insertion Sort:** Used for small runs. Also adaptive.
- **Galloping Search:** An optimization for the merge step, using binary search when one run dominates.
- **Introsort:** The hybrid sort used in C++ `std::sort`. Unstable counterpart of TimSort.

17. Practice Problems

Easy

- **Sort an Array** — LeetCode 912 (use Python's built-in sort and observe its TimSort behavior)
- **Sort Colors** — LeetCode 75 (DNF, not TimSort, but practice sorting)

Medium

- **Insertion Sort List** — LeetCode 147 (TimSort uses Insertion Sort for small runs)
- **Merge Intervals** — LeetCode 56 (sorting is the first step)

Hard

- **Implement TimSort** — GFG (full implementation)
- **Sorting Analysis** — Compare TimSort, Quick Sort, Merge Sort on different data patterns

18. Interview Explanation

"TimSort is a hybrid sorting algorithm that combines Merge Sort and Insertion Sort. It's designed to perform well on real-world data by exploiting natural order. It scans the array for already sorted sequences (runs), extends short runs using Insertion Sort, and merges runs using a balanced merge strategy. It's stable, adaptive ($O(n)$ for nearly sorted data), and guarantees $O(n \log n)$ worst-case time. It's used in Python, Java, and JavaScript's standard sort functions."

19. Revision Notes

- Hybrid of Merge Sort + Insertion Sort.
- Finds natural runs, extends short ones, merges them.
- Adaptive: $O(n)$ for nearly sorted, $O(n \log n)$ worst case.
- Stable, $O(n)$ space.
- Used in Python, Java, JavaScript.
- Key: minrun (32–64), natural runs, merge invariant.

20. Final Cheat Sheet

Property	Value
When to use	General-purpose stable sort, real-world data
Type	Hybrid (Merge + Insertion), adaptive, stable
Time	$O(n)$ best, $O(n \log n)$ worst
Space	$O(n)$
Stable	Yes
Key code idea	find runs → extend short runs with Insertion Sort → merge runs with Merge Sort
Edge cases	Empty, single, sorted, reverse, duplicates, random

END OF SORTING GUIDE

This guide covers 15 sorting algorithms and concepts with implementations in C++ and Python, complexity analysis, common patterns, practice problems, and interview-ready explanations.

Key takeaway: Know **Merge Sort** and **Quick Sort** inside out for interviews. Understand **Counting Sort** and **Radix Sort** for linear-time sorting. Master **Sort + Greedy** and **Sorting Intervals** pattern. Use **TimSort** conceptually to explain why Python's sort is fast.